

MACHINE LEARNING



SARASWATI Education Society's

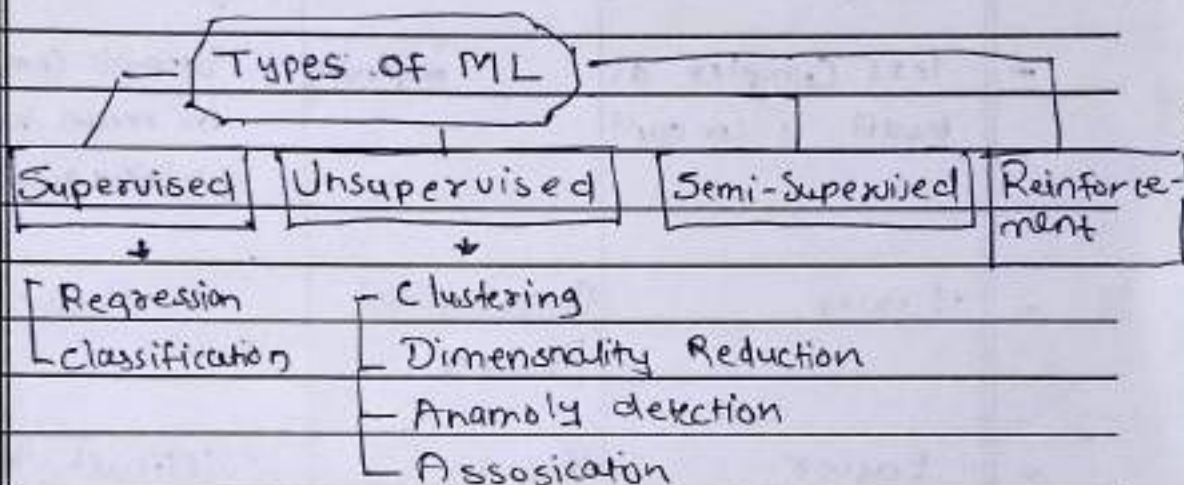
SARASWATI College of Engineering

PAGE NO.: 1

DATE:

Definition

Machine Learning is a field of computer science that uses statistical techniques to give computer systems the ability to "learn" with data, without being explicitly programmed.



Batch ML vs Online ML

• Batch ML:

Also known as offline learning, involves training a machine learning model using entire dataset at once.

In batch model is trained on a fixed dataset & parameters are updated based on gradients computed from entire dataset.

• Online ML:

Also known as incremental learning or real-time learning, involves training a ML model continuously as new data become available over time.

In online learning, model is updated incrementally with each new data point or small batch of data.



Online learning libraries

River

vopai Rabbit

Offline Learning

Features

Online Learning

- less complex as model is constant

Complexity

Dynamic Comp. as the model keep evolving over time

- fewer

Computational Power

Continuous

- Easier to implement

Use in Production

Difficult to implement.

Image classification & training in ML.

Applications

used in finance, health new data patterns.

Industry moves Tools.

Tools

Active research, New pt tools.

e.g. Skikit, TensorFlow, Pytorch, Keras, Spark Mlib.

e.g. MOA, SAMOA, skikit-multiflow



ML TYPES on the basis of Learning.

2 TYPES

Instance Based Model Based

1. Instance Based Learning

- It is also known as memory-based learning or lazy-learning.
- here, the systems that learn the training examples by heart and then generalize to new instances based on some similarity measures. It builds the hypotheses from the training instances.

Instance-based learning algorithms

1. K Nearest Neighbor (KNN)
2. Self-Organizing Map (SOM)
3. Learning Vector Quantization (LVQ)
4. Locally Weighted Learning (LWL)
5. Case-Based Reasoning ...etc.
- etc.

2. Model Based Learning

- It involves creating a model that generalizes from the training data to make predictions on new, unseen data. This model can capture patterns, relationships, and features within the data to provide a way to understand and predict outcomes.

Model-based learning algorithms are:

decision trees, linear regression, neural networks, support vector machines etc.



* Challenges in Machine Learning *

1. Data Collection
2. Insufficient Data / Labelled Data
3. Non Representative Data
4. Poor Quality Data
5. Irrelevant Features
6. Overfitting
7. Underfitting
8. Software Integration
9. Offline Learning / Deployment
10. Cost Involved

* Application of Machine Learning (B2B) Sectors of ML.

1. Retail - Amazon / Big Bazaar

- a. buying Pattern of user
- b. deciding Positioning of Product.
- c. targeted marketing
- d. Seasonal Product etc.

2. Banking and Finance

- a. Profile matching with past defaulters of Loan.
- b. Plans according to customers. etc.

3. Transport - OLA

- a. Search Pricing according to time
- b. Demand Forecasting
- c.



4. Manufacturing - Tesla

- a. Fault occurrence sensors.
- b. Predictive Maintenance.

5. Consumer Internet - Twitter

- a. ~~twits~~ Sentiment analysis
tweets

★ Machine Learning Development Life Cycle (MLDLC)

1. Frame the Problem
2. Gathering Data
3. Data Preprocessing
4. Exploratory Data Analysis
5. Feature Engineering and Selection
6. Model Training, Evaluation and Selection
7. Model Deployment
8. Testing
9. Optimize



* Various Data Based Job Roles

1. Data Engineer
2. Data Analyst
3. Data Scientist
4. ML Engineer

1. Data Engineer Job Roles

- Scrape Data from the given sources.
- move / store the data in optimal servers / warehouses.
- Build data pipelines / APIs for easy access to the data.
- Handle databases / data warehouses.

2. Data Analyst Job Roles

- Cleaning & organizing Raw data.
- Analyzing data to derive insights.
- Creating data visualizations.
- Producing & maintaining reports.
- Collaborating with teams / colleagues based on the insight gained.
- Optimizing data collection procedures.

3. Data Scientist Job Roles

- Description: "A data scientist is someone who is better at statistics than any software engineer and better at software engineering than any statistician".
- Josh Wills on Quora
- It is a fullstack developer who can do all things like data engineering, data Analytics & ML engineering.



4. ML Engineer Job Role

- Deploying machine learning models to production ready environment
- Scaling and optimizing the model for production
- Monitoring and maintenance of deployed models.

* Tensor in Machine Learning

Tensors are mathematical objects that can be used to describe physical properties, like scalars & vectors, scalar is a zero rank tensor
vector is a first rank tensor

Types of Tensors

- 0D Tensor / vector

Dimensions are zero

- 1D Tensor / vector

$[1, 2, 3, 4] \rightarrow 1D \text{ Tensor}$ 1D Tensor

$\rightarrow$ vector

2D Tensor / Matrices

Collection of vectors:

$$\begin{bmatrix} [1, 2, 3] \\ [4, 5, 6] \\ [7, 8, 9] \end{bmatrix} \Rightarrow 2D$$

ND Tensors

3D Tensor = $4 \times 3 \times 3$

4D tensor = vector of 3D tensor

5D Tensor = metrics of 4D tensor

$\vdots$



* Main Python Tools *

- Anaconda
- Jupyter Notebook
- Virtual Environment
- Kaggle Notebook
- Google Collab

End to End Project

Steps

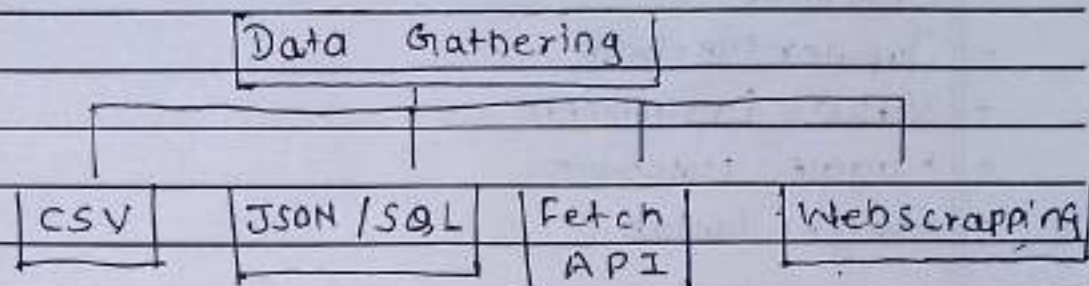
0. Preprocess + EDA + Feature Selection
1. Extract input and output cols
3. Train test split
4. Train the model
5. Evaluate the model / model selection
6. Deploy the model

* How to Frame a Machine Learning Problem

1. Business Problem to ML Problem Conversion
2. Type of Problem
3. Current Solutions Availability
4. Getting Data
5. Metrics to measure
6. Online Vs Batch ?
7. Check Assumptions



* Gathering Data



1] CSV - Comma Separated Values

Some important Commands and parameters

1. Importing pandas

`import pandas as pd`

2. Opening a local csv file

`df = pd.read_csv('file-name.csv')`

3. Opening a ~~csv~~ Tsv file.

`pd.read_csv('file-name.tsv', sep = '\t')`

4. giving column names with

`pd.read_csv('file-name.csv', name = ['col1', 'col2'])`

5. Choosing particular column index column

`pd.read_csv('file-name.csv', index_col = 'col-name')`

6. Header parameter

for starting first row in a no. of column.

7. Usecols parameter

import only required parameters.



8. squeeze parameters.
 - Convert pandas df into series object.
9. Skiprows / n rows Parameter
 - skiprows for skipping unwanted rows with give index of number.
 - n rows for importing n no. of rows
10. Encoding parameter
 - change the encoding of character
11. Skip bad lines : Parser Error
12. dtypes parameter: Change datatype of column with this adding as dictionary.
13. Handling Dates
convert as object as "datetime64"
14. Convertors
Applying transformation to any features.
15. na-values parameter
assigning / specifying any values to na values
e.g. na_values = ['-', '0']
16. Loading a huge dataset in chunks
chunksize = number
pd.read_csv('file.csv', chunksize = 5000)



> Working with JSON / SQL

- JSON - JavaScript Object Notation

For loading json data

```
pd.read_json('data.json')
```

- SQL - Structured Query Language

For loading sql data

- run sql file in any database (e.g. mysql)

- install connector library

```
!pip install mysql.connector
```

- import mysql.connector

run query:

```
Conn= mysql.connector.connect(host='localhost',
```

```
user='root',
```

```
Password='',
```

```
database='world')
```

it will return connection object
connection

* Now pass connection object on Function.

```
- pd.read_sql_query("SELECT * FROM city", Conn)
```

↓ (object)
(table-name)



* Fetching Data From an API

API - Application Programming interface

These are the data pipelines for sharing data from one application to other

For loading data use requests module.

```
- import pandas as pd
import requests
response = requests.get('url')
response.json()
```

Fetching data using Web Scraping

For web scraping use beautiful soup library.

```
import pandas as pd
import requests
import bs4
from bs4 import BeautifulSoup
web = request.get('url').text
now create object of library
soup = BeautifulSoup(web, 'xml')
```



★ Understanding Your Data ★ D-19

1. Data Size >> `df.shape`
2. Data head >> `df.head()`
3. Data sample >> `df.sample(5)` <Random>
4. Data type >> `df.info()`
5. Missing values >> `df.isnull().sum()`
6. Maths summary of data >> `df.describe()`
7. Duplicate values >> `df.duplicated().sum()`
`df.drop_duplicates()`
8. corr betⁿ cols >> `df.corr()`
`df.corr()['col-name']`

★ EDA - Univariate Analysis ★ D-20

Individual analysis of every columns.

1. Categorical Data.

a. Count plot.

>> `sns.countplot(df['col-name'])`

b. Pie Chart

>> `df['col-name'].value_counts().plot(kind='pie')`

2. Numerical Data.

a. Histogram >> `plt.hist(df['col-name'])`

b. Distplot >> `sns.distplot(df['col-name'])`

c. Boxplot >> `sns.boxplot(df['col'])`

ds. `df['col'].min(), .max(), .mean(),`
`df['col'].skew()`



EDA - Bivariate and Multivariate

D-21

Exploratory Data Analysis

Type ② Bivariate

Type ③ Multivariate

The Bivariate & Multivariate analysis is easy with seaborn library.

① Scatter plot (numerical - numerical)

`sns.scatterplot(x, y, hue, style, size)``x = numerical-datavalue, y = numerical-datavalue`

② bar plot (numerical - categorical)

`sns.barplot(x = categorical, y = numerical, hue)`

③ box plot (numerical - categorical)

`sns.boxplot(x = categorical, y = numerical, hue)`

④ Distplot (numerical - categorical)

`sns.distplot(categorical-data, numerical, kde=)`

⑤ HeatMap (categorical - categorical)

`sns.heatmap(pd.crosstab('categorical', 'numerical'))`
`'categorical'``pd.crosstab('categorical-data', 'categorical-data')`

⑥ ClusterMap (categorical - categorical)

`sns.clustermap(pd.crosstab('categorical', 'categorical'))`



⑦ Pair Plot

`sns.pairplot (data_frame, hue = )`

⑧ Line Plot (numerical - numerical)

X = time based number

Y = numerical

`sns.lineplot( x = , y = )`

★ Pandas Profiling ★

Pandas Profiling is a library which provides a solution to understanding data with the help of generating reports for datasets that have numerous features.

- Installing Pandas Profiling

`pip install pandas profiling`

- Importing

`from pandas-profiling import ProfileReport`

- Make an object & add dataframe to it
`report = ProfileReport(dataframe)`

- Make html file

`report.to_file(output_file = 'output.html')`

The generated html file will be downloaded in your folder.

Feature Engineering

Feature Transformation

Feature Construction

Feature Selection

Feature Extraction

- Missing Value Imputation
- Handling Categorical Features
- Outlier Detection
- Feature Scaling

Feature Scaling (Standardization)

* When to use Standardization :-

Algorithms

1. K-Means
2. K-NN
3. PCA
4. ANN
5. Gradient Descent

Feature scaling is a technique to standardize the independent features present in the data in a fixed range.

Standardization is also called as Z-score Normalization

$$X'_i = \frac{X_i - \bar{X}}{\sigma}$$



Feature Scaling - (Normalization)

Normalization

Normalization is a technique often applied as a part of data preparation for ml. The goal of normalization is to change the values of numeric columns in the dataset to use a common scale, without distorting difference in the range of values or losing information.

MinMax Scaling

Mean Normalization

MaxAbs Scaling

Robust Scaling

MinMax Scaling
$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

range = $[0, 1]$
min = 0, max = 1.

Mean Normalization

range = $[-1, 1]$
$$x'_i = \frac{x_i - x_{\text{mean}}}{x_{\max} - x_{\min}}$$

MaxAbs Scaling

range = $[-1, 1]$
$$x'_i = \frac{x_i}{|x_{\max}|}$$

used when sparse data

Robust Scaling

range =
$$x'_i = \frac{x_i - x_{\text{median}}}{IQR}$$

used when outliers on data



Feature Transformation

2] Handling Categorical Features

data

Numerical Categorical

Nominal

ordinal

→ one hot encoding

→ Ordinal Encoding: for independent variables

→ Label Encoding for dependent (classification) variables.

Ordinal Data

Order matters, categories have weighted magnitude.

Ordinal Encoder $\Rightarrow$ for categorical ordinal independent data (X)

Label Encoder $\Rightarrow$ For categorical ordinal dependent variables (Y) output column is classification.



Nominal Categorical Data

To convert nominal data we use
One Hot Encoding

in pandas we have inbuilt method called
`pd.get_dummies()`

In sklearn.preprocessing we have a ~~cat~~.
model/class named `OneHotEncoder`
for conversion

Dummy Variable Trap

The Dummy variable trap also called as
multicollinearity because it generates/causes
multicollinearity among the dummy or onehot encoded
variables.

To reduce multicollinearity always drop
(n-1) column from n columns which are
formed from encoding

Column Transformer

This is shortcut method for Encoding
multiple features.

Import.

```
from sklearn.compose import ColumnTransformer  
transformer = ColumnTransformer (transformers  
= [ (1), (1), (1)... ], remainder='')
```



Sk Learn - Pipelines

Pipelines chains together multiple steps so that the output of each step is used as input to the next step.

Pipelines makes it easy to apply the same preprocessing to train and test!

To use pipeline in ml we have library.

from sklearn.pipeline import Pipeline, make_pipeline

Pipeline and make_pipeline these 2 classes do same work we can use anyone among them

Syntax:

Pipeline(steps: list of tuples, ~~mem~~ memory: str or object with .joblib, verbose: bool)

e.g.

```
pipe = Pipeline([
    ('trf1', trf1),
    ('trf2', trf2)
], memory=None,
verbose=False)
```

Attributes:

1) named_steps: Bunch

Access the steps by name.



3) classes : ndarray of shape (n-classes)
The classes labels

3) n_features_in_ : int
No. of features seen during first step fit method

4) feature_names_in_ : ndarray of shape
(n_features_in_,)
Names of features seen during first step fit method

make_pipeline Syntax:

```
sklearn.pipeline.make_pipeline(*steps: list of  
estimator objects,  
memory: default = None,  
verbose: default = false)
```

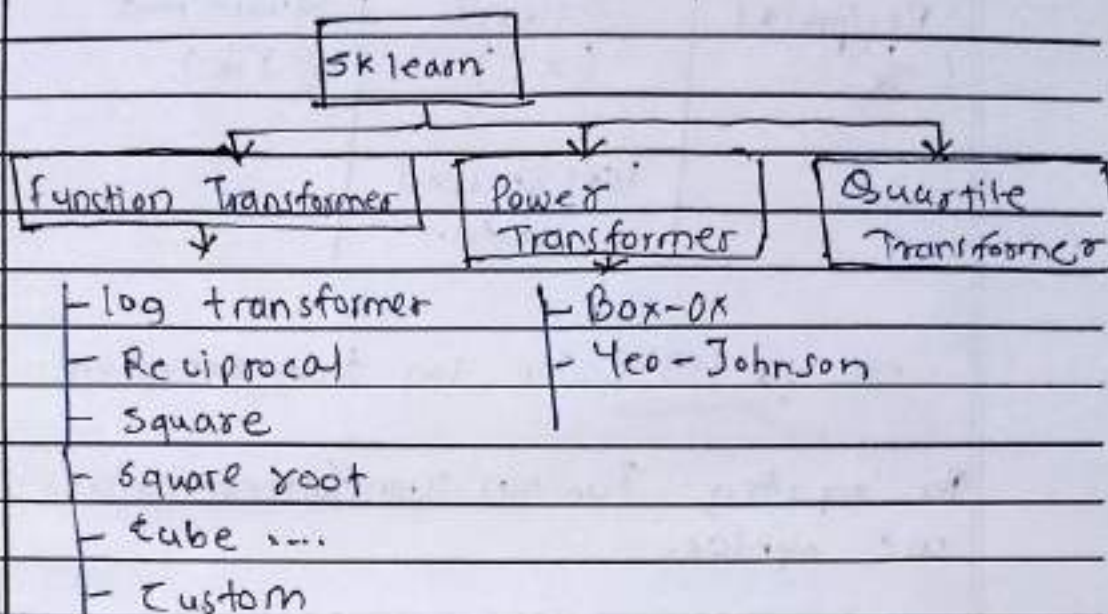
Example:

```
from sklearn.naive-bayes import GaussianNB  
from sklearn.preprocessing import StandardScaler  
from sklearn.pipeline import make_pipeline, Pipeline  
pipe = make_pipeline(StandardScaler(),  
                      GaussianNB())
```

```
pipe = Pipeline([('standardScaler', StandardScaler()),  
                 ('gaussianNB', GaussianNB())  
                 ])
```



* FUNCTION TRANSFORMER *



How To find if data is normal?

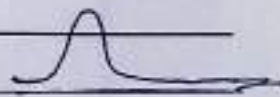
⇒ To find use
sns.distplot
Pd.skew()

• Plot the Q-Q plot

* Log Transform

Take the log of the data values
when to apply?

⇒ on right skewed data.



when not to apply?

⇒ on -ve values.

Why to use log transform.

⇒ Convert the right skewed data into normal distribution. which is helpful for linear & logistic model.



Other transfoims

Reciprocal
 $(\frac{1}{x})$

Square
 (x^2)

Square root
 $(\sqrt{x})$

left skewed
data.

use np.log1p for log transformation

For importing Function Transformer class
use syntax.

from sklearn.preprocessing import FunctionTransformer



* Power Transformer *

Box-Cox
Transformer

Yeo-Johnson
Transform

Power Transformer is another class of sklearn which is used for transformed the data into normal distribution.

Box Cox Transform

The exponent here is

$$x_i^{(\lambda)} = \begin{cases} \frac{x_i^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln(x_i) & \text{if } \lambda = 0, \end{cases}$$

The exponent here is a variable called lambda(λ) that varies over the range of -5 to 5, and in the process of searching, we examine all values of λ .

Finally, we choose the optimal value (resulting in the best approximation to a normal distribution) for your variable.

Syntax for importing.

from sklearn.preprocessing import PowerTransformer
Now make an object of it.

Pt = PowerTransformer(method='box-cox')



★ Yeo - Johnson Transform ★

if..

$$x_i^{(\lambda)} = \begin{cases} [(x_i + 1)^\lambda - 1] / \lambda & \lambda \neq 0, x_i \geq 0 \\ \ln(x_i + 1) & \lambda = 0, x_i \geq 0 \\ -[(-x_i + 1)^{2-\lambda} - 1] / (2-\lambda) & \lambda \neq 2, x_i < 0 \\ -\ln(-x_i + 1) & \lambda = 2, x_i < 0 \end{cases}$$

This transformation is somewhat of an adjustment to the Box-Cox transformation, by which we can apply it to negative numbers.

Syntax for import.

```
from sklearn.preprocessing import PowerTransformer
```

make an object of Yeo-Johnson Transform

```
pt = PowerTransformer()
```

(We don't need to specify "method" in Yeo-Johnson it is by default set on that)



Encoding Numerical Features

Discretization
(Binning)

Binarization

* Discretization

Discretization is the process of transforming continuous variables into discrete variables by creating a set of contiguous intervals that span the range of the variable's values.

Discretization is also called binning, where bin is an alternative name for interval.

Why to use Discretization:

1. To handle Outliers
2. To improve the value spread

★ Types of Discretization ★

Binning

Unsupervised
Binning

Supervised
Binning

Custom
Binning

Equal width
(Uniform)

Decision Tree

Equal frequency
(quantile)

kmeans



* Unsupervised Binning *

① Equal Width / Uniform Binning

$$\text{Formula} = \frac{\text{Max} - \text{Min}}{\text{bins}}$$

• Characteristics

1. Handle Outliers for some spread
2. No change in spread.
3. Bins are equally distributed.

② Equal Frequency / Quantile Binning

$$\text{Intervals} = 10$$

Each interval contains 10% of total observations

Characteristics

1. Handle Outliers
2. Change in value spread
3. Value is equally spread.

③ KMeans Binning

No. of bins is decided by user K.

Use Kmeans algorithm for Binning

6

Characteristics

1. Adapts the distribution of the data
2. Cluster-Based Binning
- 3.



* Custom Binning *

Encoding discretized variable.

Sklearn



`KBinsDiscretizer()`

Bins

Strategy

Encoding

Uniform

Ordinal

Quantile

One hot encoding

Kmean

* Custom / Domain Based Binning

With the help of Pandas we can apply Custom Based Binning.

These Custom Binning can be made as per our requirement.

* Binarization

Here we convert the numerical values into Binary value (0, 1)s



Handling date time data

For doing operations on date & time we require to import necessary modules.

```
import datetime as dt
```

```
import time
```

in data if date and time is in object format then convert it into `datetime64[ns]` format for performing operations. using pandas function "`to-datetime()`"

e.g.

```
date['column'] = pd.to_datetime(date['column'])
```

(here, date is a dataframe)

There are different - different datetime functions in a datetime module.

```
import datetime as dt
```

```
dt.time
```

```
dt.hour
```

```
dt.year
```

```
dt.minute
```

```
dt.month
```

```
dt.second
```

```
dt.day
```

```
dt.date
```

```
dt.month-name()
```

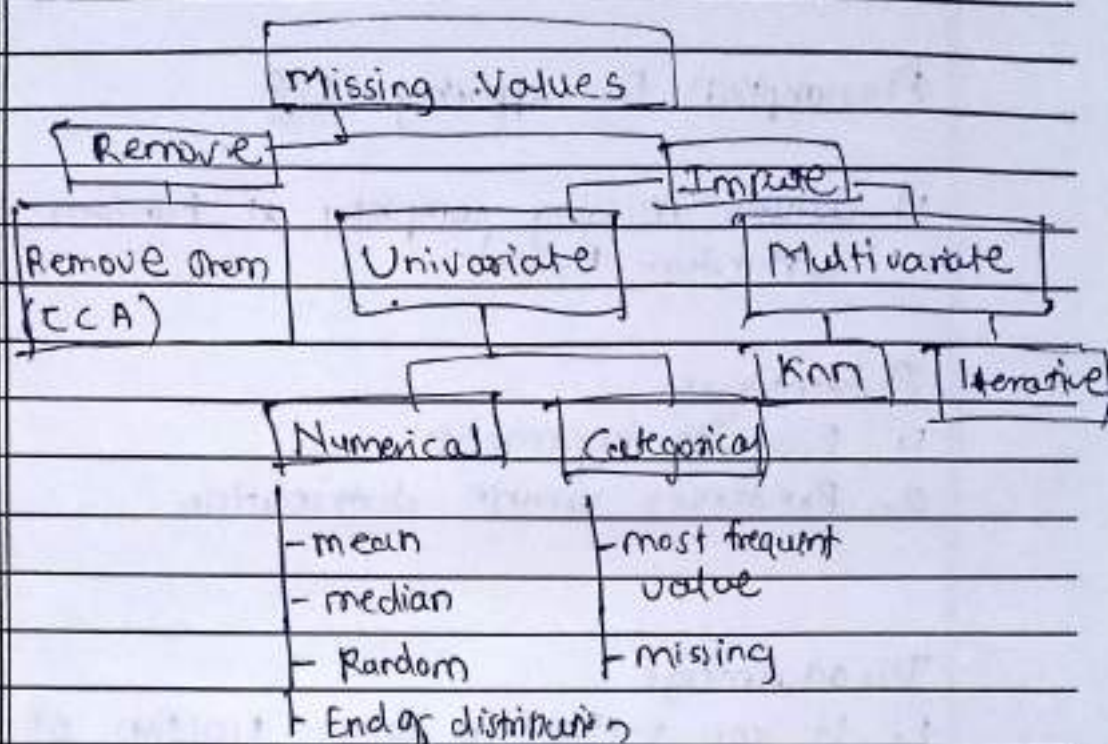
```
dt.day-name()
```

```
dt.dayofweek
```

```
dt.isocalender().week
```

```
dt.quarter
```

```
dt.today()
```



Complete Case Analysis

Complete - case analysis (CCA), also called "list-wise deletion" of cases, consists in discarding observations where values in any of the variables are missing.

Complete Case Analysis means literally analyzing only those observations for which there is information in all of the variables in the dataset.

Observation = Row

Variables = Column



Assumption for applying CCA

- 1] Values missing completely at Random locations.
(Random Rows)

Advantages:

1. Easy to implement
2. Preserves variable distribution

Disadvantage:

1. It can exclude a large fraction of the original dataset.
2. Excluded observations could be informative for the analysis.
3. When using our models in production, the model will not know how to handle missing data.

When to Apply CCA

1. MCAR
2. Missing values = 5% of data



Handling Missing Numerical Data

Univariate
imputation

Multivariate
imputation

① Mean / median Imputation

Benefit

1) simple

2) $(5\% >)$ ~~small~~ data values which are missing

Disadvantage

1) Distribution change

2) outliers

3) Covariance / Correlation changes

When to use

1) MCAR

2) $5\% <$

② Arbitrary Value Imputation

Replace missing Numeric value with any unusual value $\{ -1, 99, 999 \dots \text{etc} \}$

Benefit : Easy to apply

Disadvantage

1) PDF distribution

2) Variance / Covariance / Correlation changes

When to apply : When Data is not missing at Random position //



④ End of Distribution Imputation.

It is an extension of arbitrary value imputation.

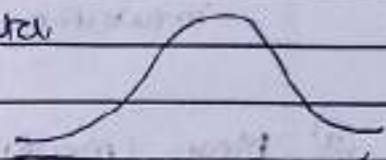
For Normally Distributed Data

it fill the missing values

with $(\text{mean} + 3\sigma)$

$(\text{mean} - 3\sigma)$

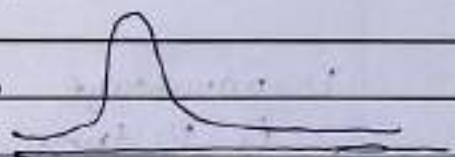
where σ is a sigma. S.D.



For Skewed Distributed Data

it fill the missing values

with $(IQR \text{ proximity})$



$$Q_2 = -1.5 * IQR$$

$$Q_3 = +1.5 * IQR$$

$$Q_1 = 25^{th}$$

$$Q_3 = 75^{th}$$

Benefit

1) easy to use.

Disadvantage:

1) PDF

2) Covariance

3) Variance

When to use

1) Data is Not Missing at Random
NMAR



* Handling Categorical Missing Data *

Most frequent Value
Imputation

Missing Category
Imputation

* Most Frequent Value Imputation

MFVI involves replacing missing values with most frequently occurring value (mode) in column.

To import Syntax

```
from sklearn.impute import SimpleImputer  
make an object
```

```
imputer = SimpleImputer(strategy='most-frequent')  
imputer.fit_transform(X_train)
```

* Missing Category Imputation

MCI is a technique used primarily for categorical variables. It involves creating a new category (label) specifically for missing values.

To implement with sklearn (Syntax)

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy='constant',  
                        fill_value='Missing')
```

```
imputer.fit_transform(X_train)
```

```
imputer.transform(X_test)
```



Random Sample Imputation

Missing Indicator

Automatic selection of parameter using Grid search CV.

1. Random Imputation

⇒ a Preserves the variance of the variable

b Memory heavy for deployment, as we need to store the original training set to extract values from and ~~per~~ replace the NA in coming observations.

c Well suited for linear models, as it does not distort the distribution, regardless of the % of NA.

2. Missing Indicator

Replaces the NA with True & present value with false in new column. So model can ignore accuracy.

import

from sklearn.impute import MissingIndicator

mi = MissingIndicator()

mi.fit(X_train)

Drive method is inbuilt in SimpleImputer

si = SimpleImputer(add_indicator=True)

True the add-indicator parameter in SimpleImputer



3. Automatically-select-imputer - parameter using GridSearchCV.

GridSearchCV is a sklearn class in model-selection which is used for different-value-parameter computation.

```
import
```

```
from sklearn.model-selection import train-test-split,  
GridSearchCV
```

* Multivariate Imputation technique.

- The KNN imputer is a technique used in multivariate imputation to fill missing values by considering k of their k -nearest neighbors.

```
import
```

```
class sklearn.impute.KNNImputer (*, missing=  
    values = nan, n-neighbors = 5,  
    weights = 'uniform', metric = 'euclidean',  
    add-indicator = False,  
    keep-empty-features = False)
```

• Iterative Imputer / MICE

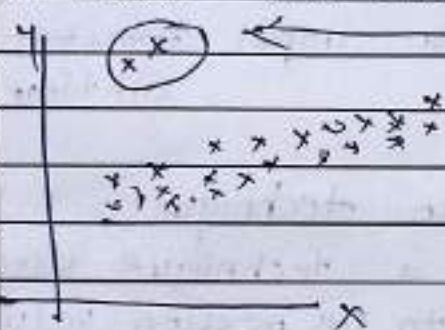
MICE stands for Multivariate Imputation by Chained Equations.

It is a method for finding missing values by modeling each variable with missing data as a function of other variables. It is an iterative process that continues till convergence.



Outliers

Outliers are data points significantly different from the majority in a dataset. In ML, outliers can impact model performance by skewing results or introducing noise. Identifying & handling outliers is crucial for building accurate & robust ML models.



These are outliers which are different from the majority of data.

Bad Impact of Outliers on ML algorithms.

- Linear
- Logistic
- Adaboost
- Deep Learning

Non-effective ML algorithms.

- Tree based

How to Treat Outliers.

- (1) Trimming (Removing directly)
- (2) Capping (setting boundaries)
- (3) Treat accordingly like it is NaN value.
- (4) Do Discretization



How To Detect Outliers ?

1. Normal Distribution

if observation is $(\mu + 3\sigma)$ & $(\mu - 3\sigma)$
out from these 2 Range's. we treat
that values as outliers.

2. Skewed Distribution

Detect using plotting box plot.

if No. is less than $Q_1 - 1.5 IQR$

No. is greater than $Q_3 + 1.5 IQR$

then consider that value as outliers.

3. Percentile Based Approach

if observation is less than 2.5%

if observation is greater than 97.5%

then consider that value as outliers.

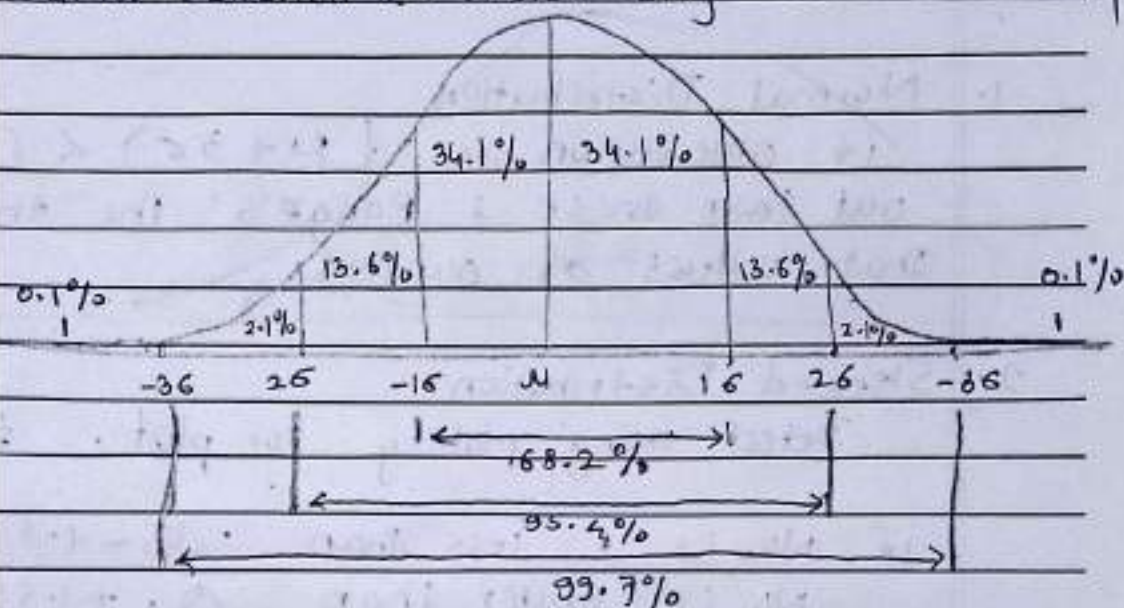
* Techniques for Outlier Detection & Removal

1. Z-score

2. IQR based filtering

3. Percentile

4. Winsorization

Outlier Detection & Removal using Z-score method

Normal Curve (Bell curve)

Z score follows Normal curve distribution
Range of value distribution

$$\left. \begin{array}{l} \mu + \sigma \\ \mu - \sigma \end{array} \right\} 68\%$$

$$\left. \begin{array}{l} \mu + 2\sigma \\ \mu - 2\sigma \end{array} \right\} 95\%$$

$$\left. \begin{array}{l} \mu + 3\sigma \\ \mu - 3\sigma \end{array} \right\} 99\%$$

Outlier Treatment

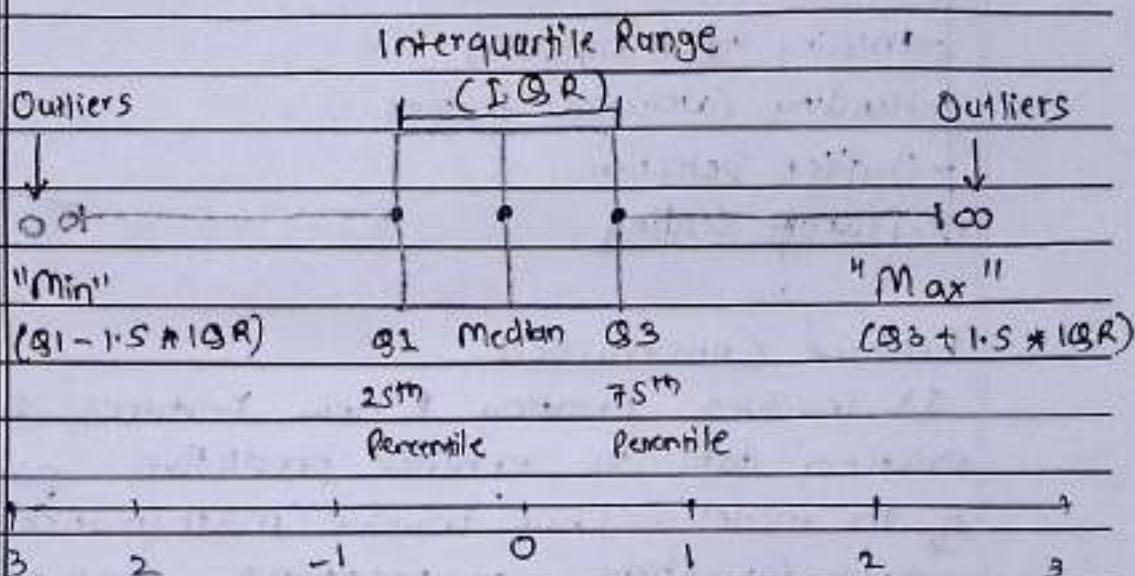
Trimming
directly remove the outliers

Capping
set an boundary to lower & upper limit for outliers



* Outlier Detection & Removal using IQR Method

The IQR Method is used at the time when our data is skewed on right or left side



IQR Proximity Rule //

$$IQR = Q3 - Q1$$

$$75th \text{ percentile} - 25th \text{ percentile}$$

- Outlier Detection Using Percentile Method
For applying Percentile method there is no condition is mention we can apply on any type of data distribution.

In this method we set the equal amount of boundary on upper & lower limit in percentile.

most common is 1 percentile & 5 percentile.



Feature Construction

SARASWATI Education Society's

SARASWATI College of Engineering

PAGE NO.: 41

DATE: 2/7/24

Feature Engineering

Feature transformation Feature Construction Feature Selection Feature Extraction

- Missing Value Imputation
- Handling Categorical features
- Outlier Detection
- Feature Scaling

Feature Construction

It involves creating new features from existing data to enhance predictive power of the model. This involves mathematical transformations, aggregations, combinations of existing features, & domain-specific knowledge to generate features.

Feature Splitting

Feature splitting involves breaking down a single feature into multiple components, which can provide more granular information and potentially improve model performance.

Example:

① Splitting a timestamp into year, month, day, hour etc.

② Splitting text features
Splitting Categorical features into binary (dummy) variables.



Feature selection & Extraction in COO

Curse of Dimensionality

Curse of features

The curse of dimensionality is a phenomenon that occurs when analyzing & organizing data in high-dimensional spaces.

Key aspects:

1. Volume Increase : A dimension increases the volume of the space. grows exponentially.
2. Sparsity : Data point become sparse. The distance between data points increases.
3. Increased Computational Complexity : requires more computational resources to process.
4. Overfitting in ML models.

To Reduce the Curse of Dimensionality we use

Dimensionality Reduction Techniques

Feature Selection



- ① Forward selection
- ② Backward Selection
- ③ Bi-directional selection

Feature Extraction



- 1) PCA
- 2) LDA
- 3) t-SNE



Principle Component Analysis (PCA)

PCA converts the higher dimensional data into best possible lower dimensional data.

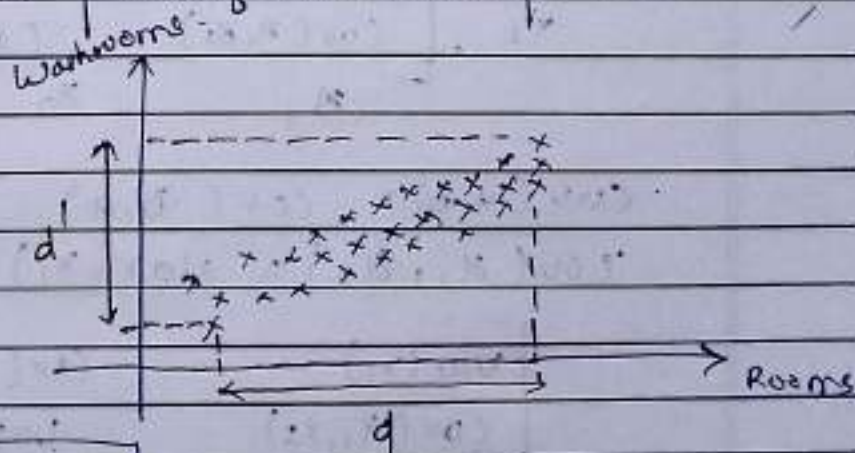
Feature Extraction is an updation over a feature selection's limitations.

Benefits of PCA

- 1) Faster execution of algorithm.
- 2) Visualization becomes very easy
PCA converts the higher dimensional data into 3D or 2D. if
eq. 10 D dim. data $\rightarrow$ PCA $\rightarrow$ 3D data

* If the data contains the columns in which variance is equally spreaded. then the feature selection technique does not work at this time we use PCA. For better understanding see the below fig of equally variance spread & holding in 2 columns.

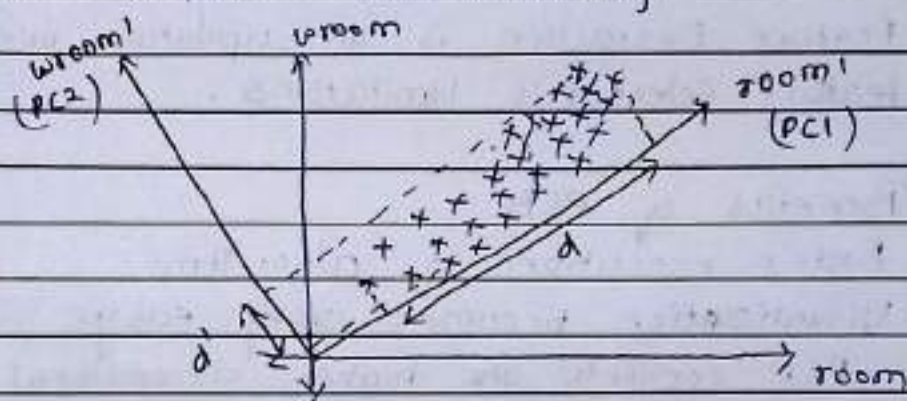
No. of rooms | No. of workrooms | Price.



$$d = d'$$



According to the data spread the PCA algo creates new axis at new plane which reduces the any one's dimension among 2 columns the new axis columns called as PC1 & PC2. see below fig.



The No. of PC is n

where n is no. of original features in data.

* Key Points:

- Covariance and Covariance matrix
- Covariance is a relation between 2 columns.

Covariance matrix

$$\begin{matrix} x_1 & & \\ x_2 & & \end{matrix} \begin{bmatrix} \text{Cov}(x_1, x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_2, x_1) & \text{Cov}(x_2, x_2) \end{bmatrix} \begin{matrix} x_1 \\ x_2 \end{matrix} \quad 2 \times 2$$

$$\text{Cov}(a, b) = \text{Cov}(b, a)$$

$$\text{Cov}(x_1, x_1) = \text{Var}(x_1)$$

$$\begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_2, x_1) \\ \text{Cov}(x_1, x_2) & \text{Var}(x_2) \end{bmatrix} \quad 2 \times 2$$



Similarly for 3×3 , $x|y|z$

$$\begin{array}{c|ccc} x & V_x & C_{x,y} & C_{x,z} \\ y & C_{y,x} & V_y & C_{y,z} \\ z & C_{z,x} & C_{z,y} & V_z \end{array}$$

$x \quad y \quad z$

- Eigen vectors.

These are special vector. The direction of eigen vectors is never changed while applying transformation. only magnitude changes direction remains same in eigen vector. $A\vec{V} = \lambda\vec{V}$, always true.

- Eigen values.

These are One measurement of how much the eigen vector is stretched or shrunk after transformation

measures the amount of stretching or shrinking of eigen vector.

Problem:

Find a unit vector which maximizes the variance.

The largest eigenvector of covariance matrix always points into the direction of largest variance of data.



* STEPS TO SOLVE PCA

Step 1: mean centring of the whole data

Step 2: Find covariance matrix

Step 3: Find the eigen value & eigen vectors for the covariance matrix. ($\lambda_1, \lambda_2, \lambda_3$ are eigen values)

Step 4: After getting Principle Components transform all the points from higher dim space to lower w.r.t. Principle Component.

for transforming points

firstly do the (dot) product of $U^T \cdot X$ with all points one by one on principle Component and take transpose of the data shape.

$(1000, 3)$ $(2, 3)$
 ↓ ↓ ↓ ↓
Row Col vectors 3D space

$1000, 3$ $(3, 2)$ (Transpose of vector)
 ↘ ↗
 (1000, 2) //

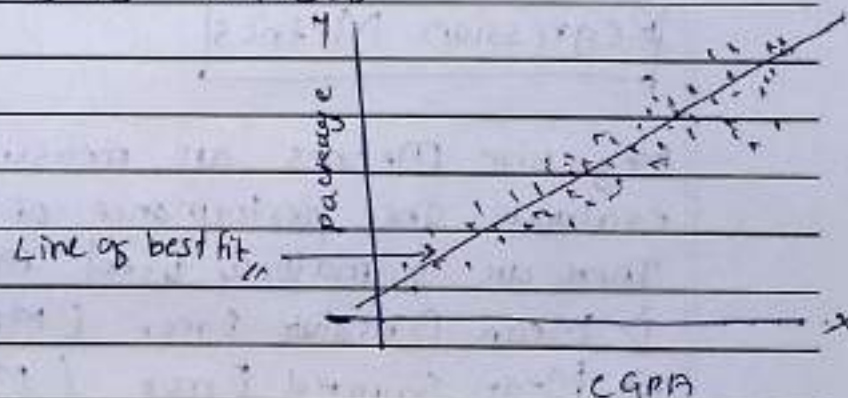


Simple Linear Regression

The Simple LR is a supervised learning method. The LR algorithm draws a line of best fit on a linear sort of data.

formula $\cdot$ $y = m \cdot x + b$

where y is dependently variable,
 x is independent var.
 m is weightage
 b is offset



import

~~from sklearn.model_selection~~

from sklearn.linear_model import LinearRegression

make object

lr = LinearRegression()

$$b = \bar{y} - m\bar{x}$$

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$



Equation of Line of best fit.

$$y = m\bar{x} + b$$

where, $\bar{y}$ = mean of y_i

$\bar{x}$ = mean of x_i

y_i = current value of y

x_i = current value of x

Regression Metrics

Regression Metrics are measures used to evaluate the performance of a regression model. There are 5 majorly used metrics.

- 1) Mean Absolute Error (MAE)
- 2) Mean Squared Error (MSE)
- 3) (RMSE) Root Mean Squared Error
- 4) R-squared (R^2)
- 5) Adjusted (R^2)

* **Loss Functions** : It quantifies the difference between the predicted values and the actual values for a single data point. It is also called as Error function.

MAE, MSE & RMSE are comes under loss functions.



1. Mean Absolute Error (MAE)

measures the average magnitude of errors treating all errors equally regardless of their direction.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

or

$$= \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

Advantages,

- 1) It has always a unit of dependent variable.
- 2) Robust to outliers.

Disadvantages

- 1) Modulus graph is not differentiable

2. Mean Squared Error (MSE)

measures average of squares of the errors

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

or

$$= \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

Advantages

- Differentiable
- Use as loss function

Disadvantage

- Not Robust to Outliers.



3) Root Mean Square Error:

RMSE = $\sqrt{\text{JMSE}}$ give high weight to large errors

$$= \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}} \quad \text{or} \quad \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Advantage

Output is in same unit. of dependent var.

disadvantage

Robust to outliers

* Absolute

4) R^2 Score

It is also coined as coefficient of determination or goodness of fit.

$$R^2 = 1 - \frac{\text{SSR}}{\text{SSM}}$$

where, SSR = Sum. of Squared error Regression

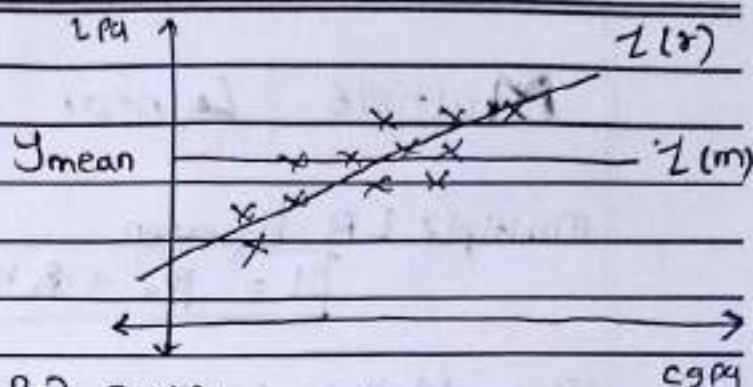
SSM = Sum of squared error. Mean

$$R^2 = 1 - \frac{\left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right]_R}{\left[\sum_{i=1}^n (y_i - \bar{y})^2 \right]_{M.}}$$

if $R^2 \rightarrow 0$ then bad

$R^2 \rightarrow 1$ then good

$R^2 \rightarrow \text{tve}$ then $\text{SSR} > \text{SSM}$ very bad.



Problem with R^2 score.

R^2 score increases with irrelevant features also R^2 score fluctuates to high or in increment ~~also~~ when any unimportant feature also added.

4) Adjusted R^2 score.

Adjust the R^2 value based on the no. of predictor in the model.

$$\text{Adjusted } R^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{(n - 1 - k)} \right]$$

where, $n \rightarrow$ no. of rows

$k \rightarrow$ total no. of independent columns.



Multiple Linear Regression

Multiple LR Equation

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

The Multiple LR is an extension of simple LR that models the relationship between two or more independent variables & a single dependent variable by fitting linear eqⁿ to observed data.

Equation in general form.

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Where Y is independent var.

β_0 is intercept / offset.

$\beta_1, \beta_2, \beta_3, \dots, \beta_n$ are coefficient for independent variables.

$x_1, x_2, \dots, x_p$ are independent variables.

ϵ is a error term.

For understanding this algorithms in detail let's take an example.

We have data of 100 students containing 4 columns:

CGPA	IQ	graduate	LPA
x_1	x_2	x_3	Y

(100, 4)

is shape of data

Eqⁿ of hyperplane is -

$$\hat{Y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

where $\hat{Y}$ is Y predicted.



$$\hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_{100} \end{bmatrix} = \begin{bmatrix} \beta_0 & \beta_1 X_{11} & \beta_2 X_{12} & \beta_3 X_{13} \\ \beta_0 & \beta_1 X_{21} & \beta_2 X_{22} & \beta_3 X_{23} \\ \vdots & \vdots & \vdots & \vdots \\ \beta_0 & \beta_1 X_{1001} & \beta_2 X_{1002} & \beta_3 X_{1003} \end{bmatrix}$$

for n rows and m columns.

$$\hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} \beta_0 & \beta_1 X_{11} & \beta_2 X_{12} & \beta_3 X_{13} & \dots & \beta_m X_{1m} \\ \beta_0 & \beta_1 X_{21} & \beta_2 X_{22} & \beta_3 X_{23} & \dots & \beta_m X_{2m} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \beta_0 & \beta_1 X_{n1} & \beta_2 X_{n2} & \beta_3 X_{n3} & \dots & \beta_m X_{nm} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & X_{11} & X_{12} & X_{13} & \dots & X_{1m} \\ 1 & X_{21} & X_{22} & X_{23} & \dots & X_{2m} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & X_{n1} & X_{n2} & X_{n3} & \dots & X_{nm} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix}$$

$A \text{ or } X$

β

$$\boxed{\hat{Y} = X \cdot \beta} \quad \text{--- (1)}$$

where X is matrix & β is coefficient matrix

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad e = y - \hat{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$



$$e = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

$$E = e^T e$$

$$\begin{matrix} [(y_1 - \hat{y}_1) & (y_2 - \hat{y}_2) & \dots & (y_n - \hat{y}_n)] & \begin{matrix} (y_1 - \hat{y}_1) \\ (y_2 - \hat{y}_2) \\ \vdots \\ (y_n - \hat{y}_n) \end{matrix} \\ (1 \times n) & & & & (n \times 1) \end{matrix}$$

$$(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 + \dots + (y_n - \hat{y}_n)^2$$

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\therefore (A+B)^T = A^T + B^T$$

$$\therefore (A-B)^T = A^T - B^T$$

$$E = e^T e = (y - \hat{y})^T (y - \hat{y})$$

$$= (y^T - \hat{y}^T) (y - \hat{y})$$

$$= [y^T - (X\beta)^T] (y - X\beta)$$

$$\dots \text{from (i)} \quad \hat{y} = X\beta$$

$$E = y^T y - y^T X\beta - (X\beta)^T y + (X\beta)^T X\beta$$

$$E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$$

$$\text{From } y^T X\beta = \beta^T X^T y$$

$$A^T B = B^T A$$



differentiate the eqn

$$\frac{dE}{d\beta} = \frac{d}{d\beta} [Y^T Y - 2Y^T X \beta + \beta^T X^T X \beta] = 0$$

$$= 0 - 2Y^T X + \frac{d}{d\beta} [\beta^T X^T X \beta] = 0$$

$$= -2Y^T X + 2X^T X \beta^T = 0$$

$$= X^T X \beta^T = Y^T X$$

$$\beta^T = Y^T X \cdot 1$$

$$[X^T X]$$

$$\beta^T = Y^T X (X^T X)^{-1}$$

$$(\beta^T)^T = [Y^T X (X^T X)^{-1}]^T$$

$$\beta = [(X^T X)^{-1}]^T (Y^T X)^T$$

$$\beta = [(X^T X)^{-1}]^T X^T Y$$

$$\beta = (X^T X)^{-1} X^T Y$$

$$\boxed{\beta = (X^T X)^{-1} X^T Y}$$

↓

↓

$$\begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} = \begin{bmatrix} (m+1) \times (m+1) \\ (m+1) \times n \\ n \times 1 \end{bmatrix} \begin{bmatrix} (m+1) \times n \\ n \times 1 \end{bmatrix}$$

$$(m+1) \times 1$$

$$(m+1) \times 1 = (m+1) \times 1$$

$$\beta$$

$$(X^T X)^{-1} X^T Y$$

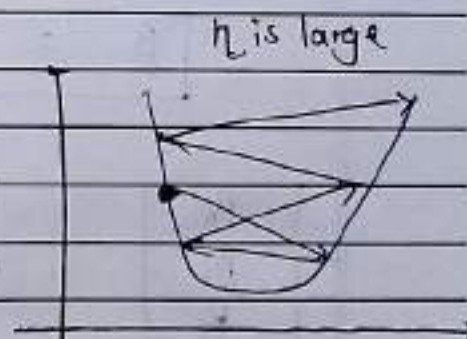
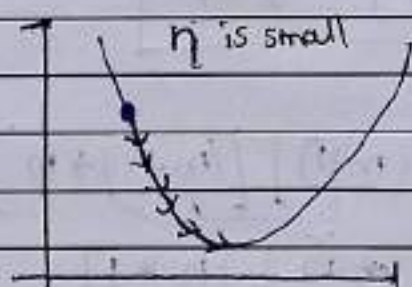


Gradient : It is a derivative that defines the effects on outputs of the function with a little bit of variation in inputs.

Gradient descent : is a method for unconstrained mathematical optimization.

It is a first order iterative algorithm for finding a local minimum of a differentiable multivariate function.

Cost

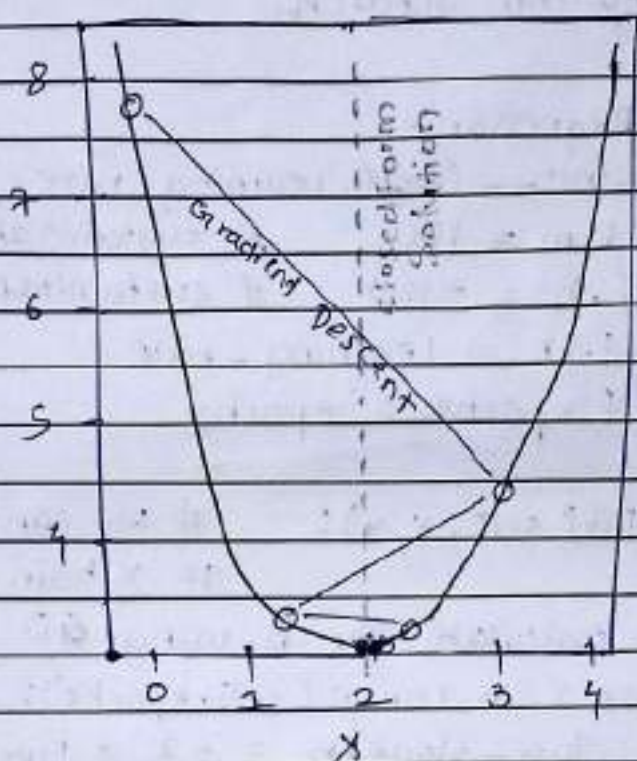


(η) is learning rate.

$$b_{\text{new}} = b_{\text{old}} - \text{slope}$$



$$b_{\text{new}} = b_{\text{old}} - \eta \text{ slope}$$



$$\text{Gradient descent} = -2 \sum_{i=1}^n (y_i - mx_i - b)$$

Formula in code

```
for i in range(self.epochs):
```

```
    loss_slope_b = -2 * np.sum(y - self.m * X.ravel()  
                                self.b)
```

```
    loss_slope_m = -2 * np.sum(y - self.m *  
                                X.ravel() - self.b) *  
                                X.ravel()
```

```
    self.b = self.b - (loss_slope_b * self.lr)
```

```
    self.m = self.m - (loss_slope_m * self.lr)
```



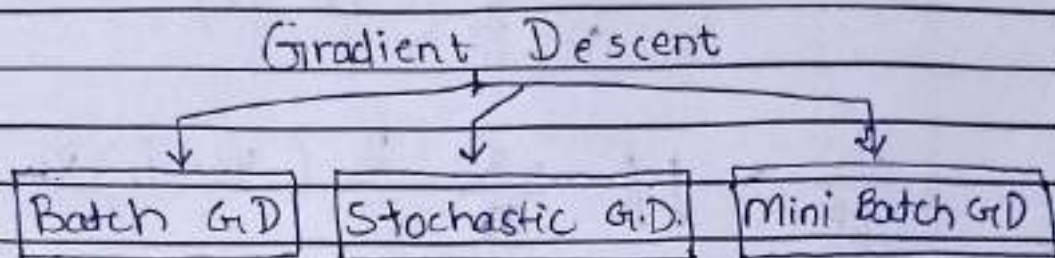
Own Algo from scratch.

line.

```
1 class GDRegressor:
2     def __init__(self, learning_rate, epochs):
3         self.m = 100 # customizable
4         self.b = -120 # customisable
5         self.lr = learning_rate
6         self.epochs = epochs
7
8     def fit(self, X, y): # we can replace with
9                         # X=train, Y=train.
10        # calculate the b using GD
11        for i in range(self.epochs):
12            loss_slope_b = -2 * np.sum(y -
13                                       self.m * X.ravel() - self.b)
14            loss_slope_m = -2 * np.sum((y -
15                                       self.m * X.ravel() - self.b) * X.ravel())
16
17            self.b = self.b - (loss_slope_b * self.lr)
18            self.m = self.m - (loss_slope_m * self.lr)
19            print(self.m, self.b)
20
21    def predict(self, X):
22        return self.m * X + self.b
23
24 # make object.
25 gd = GDRegressor(lr=0.001, epochs=100)
26 gd.fit(X, y) # replace with X=train, Y=train
27 gd..
28 y_pred = gd.predict(X_test)
```



* Types of Gradient Descent *



• Batch Gradient Descent //

Batch gradient descent sums the error for each point in a training set, updating the model only after all training examples have been evaluated.

This process referred as a training epoch.

Batch GD usually produces a stable error gradient & convergence.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$$\beta_0 = \beta_0 - \eta \text{slope} = \frac{\partial L}{\partial \beta_0}$$

$$\beta_1 = \beta_1 - \eta \text{slope} = \frac{\partial L}{\partial \beta_1}$$

$$\beta_2 = \beta_2 - \eta \text{slope} = \frac{\partial L}{\partial \beta_2}$$

$$\beta_n = \beta_n - \eta \text{slope} = \frac{\partial L}{\partial \beta_n}$$

Loss function & its derivatives of each epochs -



Loss function for MSE

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{n} \sum_{i=1}^n -2 (y_i - \hat{y}_i) X_{i1}$$

$$\frac{\partial L}{\partial \beta_2} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{i2}$$

$$\frac{\partial L}{\partial \beta_3} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{i3}$$

General formula of Loss function.

$$\boxed{\frac{\partial L}{\partial \beta_m} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{im}}$$

Formula for intercept (β_0) derivative.

$$\boxed{\frac{\partial L}{\partial \beta_0} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)} //$$



• Stochastic Gradient Descent (SGD) •

Stochastic gradient descent (SGD) runs a training epoch for each example within the dataset and it updates each training example's parameters one at a time. Since you only need to hold one training example, they are easier to store in memory. These make SGD faster. - Its frequent updates can result in noisy gradients, but this can also be helpful in escaping the local minimum & finding global one.

Intercept (β_0)

$$\frac{\partial L}{\partial \beta_0} = -2 (y_i - \hat{y}_i)$$

where, $i = \text{idx}$

Coefficient (β_1)

$$\frac{\partial L}{\partial \beta_1} = -2 (y_i - \hat{y}_i) \cdot x_i$$
$$-2 [(y_i - \hat{y}_i), x_i]$$

When no of epochs are fixed in same size for batch & stochastic

comparing with the speed

batch computes faster than stochastic

When to use Stochastic GD

- 1) Big data
- 2) Non convex function





• Mini-Batch Gradient Descent.

Mini-batch gradient descent combines concepts from both batch gradient descent and stochastic gradient descent. It splits the training dataset into small batches and performs updates on each of those batches. This approach strikes a balance between the computational efficiency of batch gradient descent & the speed of stochastic gradient descent.

Min batch = Batch GD + Stochastic GD



* Polynomial Regression

Polynomial regression is a form of regression analysis in which the relationship between the independent variable X and the dependent variable Y is modeled as an n th degree polynomial in X .

Polynomial Regression is a special case of Linear Regression where we fit the polynomial equation on the data with a curvilinear relationship between the dependent and independent variables.

$$\text{Simple LR} \Rightarrow Y = \beta_0 + \beta_1 X_1$$

$$\text{Multiple LR} \Rightarrow Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

$$\text{Polynomial LR} \Rightarrow Y = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \dots + \beta_n X_1^n$$

Cost Function : CF of Polynomial is Mean Square Error,

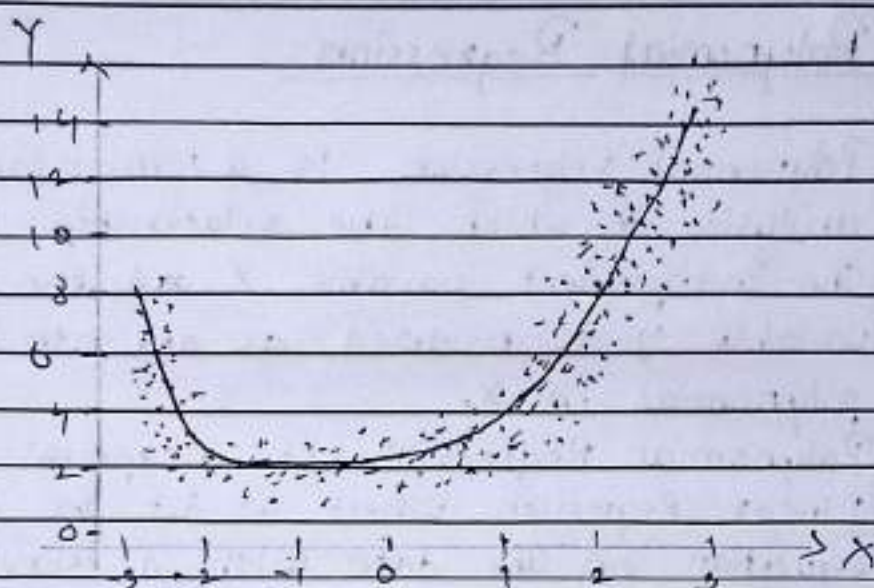
$$J = \frac{1}{n} \sum_{i=1}^n (\text{pred}_i - y_i)^2$$

Cost Function of Polynomial Reg.

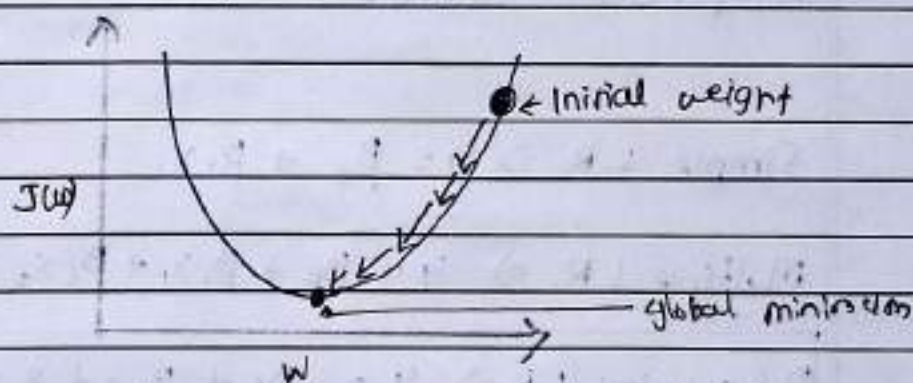
$$J = 1/n * \text{sum}(\text{square}(\text{pred} - y))$$

another form,

$$J = 1/n * \text{sum}(\text{square}(\text{pred} - (\beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_n X^n)))$$



Polynomial regression.

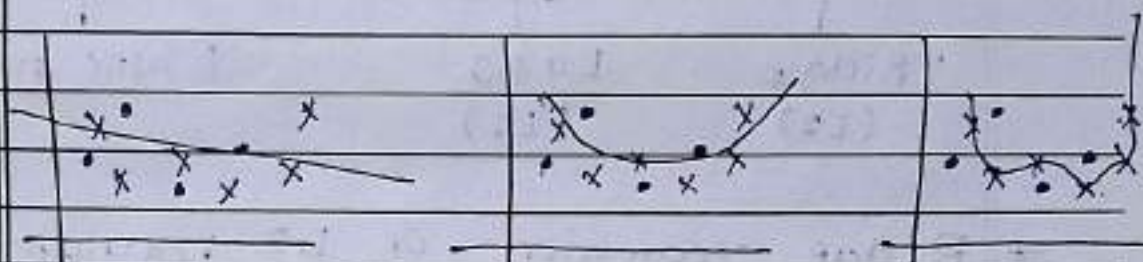


Gradient Descent for Polynomial Reg.



BIAS VARIANCE TRADE-OFF

Bias-variance tradeoff describes the relationship betⁿ a model's complexity. it trying to minimize the 2 sources of error which are bias & variance error



Model 1

Model 2

Model 3

Bias	High	Mid Low	Low
Variance	Low	Low	High
Fitting	Underfitting	best fit	Overfitting
Model	Simple	Sweet Spot	Complex

- High variance means model makes strong assumptions about the data leads to underfitting.
- High Variance means model is very sensitive to small fluctuations in the training data, leads to overfitting.

For Handling Bias Variance Trade-off: 3

- 3 important techniques.
- 1) Bagging
 - 2) Boosting
 - 3) Regularization



Regularization

Regularization is a set of methods for reducing overfitting in ML models.

Regularization

Ridge
(L2)

Lasso
(L1)

Elastic net

* Ridge regression - or L2 regularization

It penalizes high-value coefficients by introducing a penalty term in SSE loss fun.

- Using Ridge in code using sklearn class.

import

from sklearn.linear_model import Ridge
make obj

$R = \text{Ridge}(\alpha = 0.01)$ # α is hyper-parameter.

train the model.

$R.\text{fit}(X_{\text{train}}, y_{\text{train}})$

Predict.

$R.\text{predict}(X_{\text{test}})$

Ridge Regression Mathematical Formulation.

$$\text{Loss Function: } (L) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda m^2$$

$$L = \sum_{i=1}^n (y_i - mx_i - b)^2 + \lambda m^2$$

$$\frac{\partial L}{\partial b} \quad \& \quad \frac{\partial L}{\partial m}$$

$$\frac{\partial L}{\partial b} = 0, \text{ from this}$$

$$b = \bar{y} - m\bar{x} \quad \text{where } \bar{y} \rightarrow y\text{-mean} \\ \bar{x} \rightarrow x\text{-mean.}$$

$$\text{Now, } L = \sum_{i=1}^n (y_i - mx_i - \bar{y} + m\bar{x})^2 + \lambda m^2$$

(replace b by $\bar{y} - m\bar{x}$)

$$\frac{\partial L}{\partial m} = 2 \sum_{i=1}^n (y_i - mx_i - \bar{y} + m\bar{x})(-x_i + \bar{x}) + 2\lambda m = 0$$

$$= -2 \sum_{i=1}^n (y_i - \bar{y} - mx_i + m\bar{x})(x_i - \bar{x}) + 2\lambda m = 0$$

$$= \lambda m - \sum_{i=1}^n [(y_i - \bar{y}) - m(x_i - \bar{x})](x_i - \bar{x}) = 0$$

$$= \lambda m - \sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x}) + m \sum_{i=1}^n (x_i - \bar{x})^2 = 0$$

$$= \lambda m - \sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x}) + m \sum_{i=1}^n (x_i - \bar{x})^2 = 0$$

$$= \lambda m + m \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})$$

$$= m = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2 + \lambda}$$



$$m = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2 + \lambda}$$

$$b = \bar{y} - m\bar{x}$$

loss function of N-Dim. Data.

$$L = (XW - Y)^T (XW - Y) + \lambda \|W\|^2$$

$$W = (X^T X)^{-1} X^T Y$$

~~$$W = (X^T X)^{-1} X^T Y$$~~

$$W = (X^T X + \lambda I)^{-1} X^T Y$$

code for Ridge Reg. for N-dim. dataset.

```
class MyRidge:
```

```
    def __init__(self, alpha = 0.1):
```

```
        self.alpha = alpha
```

```
        self.coef = None
```

```
        self.intercept = None
```

```
    def fit(self, X_train, y_train):
```

```
        X_train = np.insert(X_train, 0, 1, axis=1)
```

```
        I = np.identity(X_train.shape[1])
```

```
        I[0][0] = 0
```

```
        result = np.linalg.inv(np.dot(X_train.T,  
                                        X_train) + np.dot(self.alpha * I)).  
                    dot(X_train.T).dot(y_train)
```



```
self.coef_ = result[1:]  
self.intercept_ = result[0]
```

```
def predict(self, x_test):  
    return np.dot(self.coef_, x_test)  
        + self.intercept_
```

Our class is created Now let, test it.

```
reg = MyRidge()  
reg.fit(x_train, y_train)  
y_pred = reg.predict(x_test)  
r2_score(y_test, y_pred)  
reg.coef_  
reg.intercept_
```

Ridge Regression using Gradient Descent

Gradient Descent..

Loss function of linear reg.

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Loss function in vector form. for linear reg.

$$L = (XW - Y)^T (XW - Y)$$

Ridge Reg. Loss fun.

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda m^2$$

LF in Vector form,

$$L = (XW - Y)^T (XW - Y) + \lambda \|W\|^2$$

$$L = (XW - Y)^T (XW - Y) + \lambda \cdot W^T W$$

where,

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{13} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & x_{23} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & x_{m3} & \dots & x_{mn} \end{bmatrix} \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

$$W = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \rightarrow (n+1)$$



$$L = (xw - y)^T (xw - y) + \lambda w^T w$$

mul by $\frac{1}{2}$

$$L = \frac{1}{2} (xw - y)^T (xw - y) + \frac{1}{2} \lambda w^T w$$

$$L = \frac{1}{2} [w^T x^T x w - w^T x^T y - y^T w x + y^T y] + \frac{1}{2} \lambda w^T w$$

$$L = \frac{1}{2} [w^T x^T x w - 2 w^T x^T y + y^T y] + \frac{1}{2} \lambda w^T w$$

$$\frac{\partial L}{\partial w} = \frac{1}{2} [2 x^T x w - 2 x^T y] + \frac{1}{2} \lambda x w$$

$$\frac{\partial L}{\partial w} = \boxed{x^T x w - x^T y + \lambda w} = \left(\frac{\Delta L}{\Delta w} \right)_1$$



Key Points about Ridge Regression

1) How the coefficients get affected
(~~B~~)

If λ value keep 0 then act as linear reg.
If λ value is increase from 0 to greater number
all the coefficients will get shrinked get
decreased towards zero but never turn
zero.

2) Higher values are impacted more.

While increasing λ towards greater value (∞)
the higher values will decreased faster than
lower lesser values. but not closed to zero
higher value = higher impact.

3) Bias Variance Trade off.

Bias & Variance both affect by λ value.
if $\lambda \Rightarrow 0$ then ~~to~~ bias $\downarrow$ overfitting.

$\lambda \Rightarrow \infty$ greater then bias $\uparrow$ underfitting.

λ value should be

= ~~low~~ bias low &
varian low.



4) Impact on the Loss Function

as λ value increase the Loss function moves
towards zero. but not become zero.

5) Why called Ridge

- Hard Constraint Ridge regression is
- λ soln come on Ridge of circle / perimeter of
circle for best results. best soln for MSE
& λ value.



• LASSO REGRESSION (L1 Regularization)

Lasso regression is a regularization technique that penalizes high value, correlated coefficients. It introduces a regularization term (also called penalty term) into the (SSE) loss function. This penalty term is absolute value of the sum of coefficients.

Controlled in turn by hyperparameter λ , it reduces select feature weights to zero.

Lasso regression removes multicollinear feature from the model altogether (auto feature selection is applied)

Lasso Reg. Loss fun.

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \boxed{\lambda \cdot \|W\|_1} \rightarrow \text{Penalty term}$$

$$L = \text{MSE} + \lambda \|W\|_1$$

$$\text{where, } \|W\|_1 \Rightarrow |w_1| + |w_2| + |w_3| + \dots + |w_n|$$

>> import lasso for code from sklearn.

from sklearn.linear_model import Lasso

Lasso_model = Lasso(alpha=0.5)

Lasso_model.fit(X_train, y_train)

Lasso_model.predict(X_test)



Key point in Lasso Regression

- 1) How are coefficient affected?
 λ value $\uparrow$ Then coefficient get shrinked.
unimportant values become zero
when λ value $\uparrow$
higher impact for decrementation of higher important values (coefficients).
- 2) Higher coefficient are affected more
as λ value $\uparrow$ Higher coefficient decreased fast & at last become zero.
- 3) Impact on Bias & Variance.
 λ value $\uparrow$:
overfitting $\downarrow$ & Bias $\uparrow$ & variance $\downarrow$
~~variance $\uparrow$ & variance $\downarrow$~~
- 4) Effect of Regularization on loss function
 λ value $\uparrow$ increases
loss function curve moves towards zero.
at the end it will reach zero.
- 5) Sparsity is created by lasso reg.



Lasso reg. ie loss function.

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda |m|$$

$$(b = \bar{y} - m\bar{x})$$

$$L = \sum_{i=1}^n (y_i - mx_i - \bar{y} + m\bar{x})^2 + 2\lambda |m|$$

$$\frac{\partial}{\partial m} = \sum_{i=1}^n (y_i - mx_i - \bar{y} + m\bar{x})^2 + 2\lambda m$$

$$= 2 \sum (y_i - mx_i - \bar{y} + m\bar{x})(-x_i + \bar{x}) + 2\lambda = 0$$

$$= -2 \sum [(y_i - \bar{y}) - m(x_i - \bar{x})](x_i - \bar{x}) + 2\lambda = 0$$

$$= -\sum [(y_i - \bar{y})(x_i - \bar{x}) - m(x_i - \bar{x})^2] + \lambda = 0$$

$$= -\sum [(y_i - \bar{y})(x_i - \bar{x}) - m(x_i - \bar{x})^2] + \lambda = 0$$

$$= -\sum (y_i - \bar{y})(x_i - \bar{x}) + m \sum (x_i - \bar{x})^2 + \lambda = 0$$

$$m \sum (x_i - \bar{x})^2 = \sum (y_i - \bar{y})(x_i - \bar{x}) - \lambda$$

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) - \lambda}{\sum (x_i - \bar{x})^2}$$

$$\sum (x_i - \bar{x})^2 \quad // \quad //$$

for

$$m > 0$$

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) - \lambda}{\sum (x_i - \bar{x})^2}$$

$$\sum (x_i - \bar{x})^2$$

for $m = 0$

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x})}{\sum (x_i - \bar{x})^2}$$

$$\sum (x_i - \bar{x})^2$$

for $m < 0$

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) + \lambda}{\sum (x_i - \bar{x})^2}$$

$$\sum (x_i - \bar{x})^2$$



Now for sparsity let's take an example.

assume: $(y_i - \bar{y}) = y$

$$(x_i - \bar{x}) = x$$

for $m > 0$.

$$m = \frac{yX - \lambda}{x^2}$$

supposed, $yX = 100$

$$x^2 = 50$$

$$m = \frac{100 - \lambda}{50}$$

for $\lambda = 0$ $m = \frac{100 - 0}{50} = \frac{100}{50} = 2$.

$$\boxed{m=2}$$

for $\lambda = 10$ $m = \frac{100 - 10}{50} = \frac{90}{50} = \boxed{\frac{9}{5}}$

for $\lambda = 50$ $m = \frac{100 - 50}{50} = \frac{50}{50} = \boxed{1}$

for $\lambda = 100$ $m = \frac{100 - 100}{50} = \frac{0}{50} = \boxed{0}$

when $yX = \lambda$ then $m = 0$

when λ values goes in -ve then we get $m < 0$ eqn. which is ~~not~~ yX .

$$m = \frac{yX + \lambda}{x^2} \text{ which again}$$

increase the m value to a +ve greater so algo stops at ~~that~~ $p = 0$



Now, for
 $m < 0$.

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) + \lambda}{\sum (x_i - \bar{x})^2}$$

$$m = \frac{xy + \lambda}{x^2} \quad \left. \begin{array}{l} xy = -100 \\ x^2 = 50 \end{array} \right\}$$

$$\text{When } \lambda = 0, \quad \frac{-100 + 0}{50} = -2 \quad \boxed{m = -2}$$

$$\lambda = 50, \quad \frac{-100 + 50}{50} = -1 \quad \boxed{m = -1}$$

$$\lambda = 100, \quad \frac{-100 + 100}{50} = 0 \quad \boxed{m = 0}$$

$$\lambda = 150, \quad \frac{-100 + 150}{50} = \frac{50}{50} = 1 \quad \boxed{m = 1}$$

for $\lambda = 150$, m become 1 so, $m > 0$.
or $m \neq 0$.

So, we will use $m > 0$ formula.

$$m = \frac{xy - \lambda}{x^2} = \frac{-100 - 150}{50} = \frac{-250}{50}$$

$$= \frac{-250}{50} = \boxed{-5}$$

$\boxed{m = -5}$, so we can't go again for
larger value than -2 so algo stops
at zero 0

2) why Ridge Reg not create sparsity.
Ridge Reg. formula.

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x})}{\sum (x_i - \bar{x})^2 + \lambda} \quad \begin{array}{l} \text{in ridge reg.} \\ \lambda \text{ is at denominator} \end{array}$$

so it can't be zero. but in lasso
 λ is in numerator so it can be
zero. cause denominator can't be
zero.



* ElasticNet Regression

- Elastic net regression essentially combines both ridge and lasso regression but inserting both the $L1$ & $L2$ penalty terms into the SSE loss function. Elastic net insert both penalty values into the cost function equation. Elastic net ~~a~~ handles ~~net~~ addresses multicollinearity while also enabling feature selection.

Loss function L

$$L = \sum (y_i - \hat{y}_i)^2 + a \|w\|^2 + b \|w\|$$

where a & b are independent.

$a = 0.5$ & $b = 0.5$. both have equal weightage.

$$\lambda = a + b, \text{ l1-ratio} = \frac{a}{a+b}$$

when

λ & l1-ratio are hyperparameters.

~~l1-ratio = 0.9 $\Rightarrow$ 90% ridge & 10% lasso~~

~~90% ridge & 10% lasso~~

~~l1-ratio = 0.4 $\Rightarrow$ 40% ridge & 60% lasso~~

l1-ratio = 0.9 $\Rightarrow$ 90% lasso & 10% ridge.

l1-ratio = 0.4 $\Rightarrow$ 40% lasso & 60% ridge.

implement elasticnet using sklearn.

from sklearn.linear_model import ElasticNet

reg = ElasticNet(alpha = 0.005, l1-ratio = 0.9)

reg.fit(X_train, y_train)

reg.predict(X_test)



Prerequisite for applying logistic regression.

- Data must be linearly separable.

Perceptron Trick

Log Reg eqⁿ $Ax + By + C = 0$

$$Ax_1 + Bx_2 + C = 0$$

$$Ax_1 + Bx_2 + Cx_3 + d = 0$$

$$[A, B, C]$$

How to label regions.

If eqⁿ of line is to point is on

$$Ax_1 + By_1 + C > 0 \Rightarrow +v \text{ region}$$

$$Ax_1 + By_1 + C < 0 \Rightarrow -v \text{ region}$$

$$Ax_1 + By_1 + C = 0 \Rightarrow \text{on the line}$$



Algo:

$$Ax + By + C = 0$$

$$W_0 + W_1x_1 + W_2x_2 = 0, \quad W_0 = C, W_1 = A, W_2 = B$$

$$W_0x_0 + W_1x_1 + W_2x_2 = 0$$

$$\sum_{i=0}^2 W_i x_i = 0 \quad [W_0 \ W_1 \ W_2] \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$



Algorithm (1)

if $x_i \in N$ and $\sum w_i x_i > 0$

$$w_n = w_0 - \eta x_i$$

if $x_i \in P$ and $\sum w_i x_i < 0$

$$w_n = w_0 + \eta x_i$$

~~where~~where, N = Negative point P = Positive point

Algorithm (2)

$$w_n = w_0 + \eta (y_i - \hat{y}_i) x_i$$

	$y_i - \hat{y}_i$	y_i	$\hat{y}_i$
case 1 = $w_n = w_0$	0	1	1
case 2 = $w_n = w_0$	0	0	0
case 3 = $w_n = w_0 + \eta x_i$	1	1	0
case 4 = $w_n = w_0 - \eta x_i$	-1	0	1

code for perceptron trick.

def perceptron(X, y):

 $X = np.insert(X, 0, 1, axis=1)$

weights = np.ones(X.shape[1])

lr = 0.1

for i in range(1000):

 $j = np.random.randint(0, 100)$ $y_hat = step(np.dot(X[j], weights))$ $weights = weights + lr * (y[j] - y_hat) * X[j]$

return weights[0], weights[1:]



Logistic Regression.

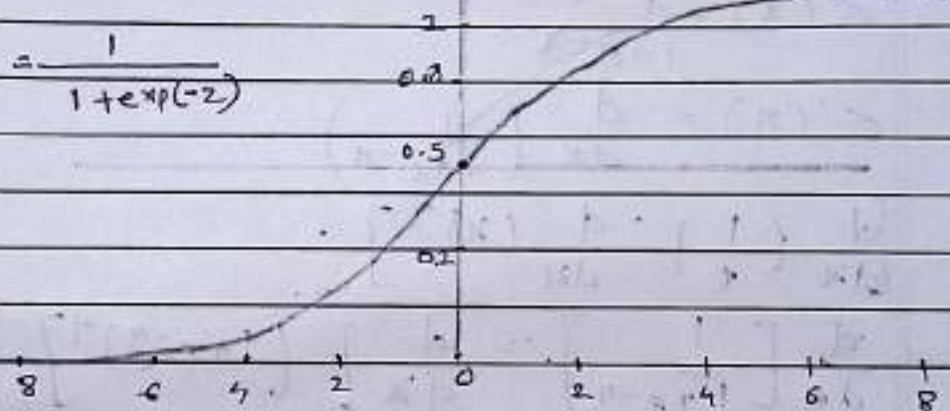
For logistic reg. we have to make changes in our Algo. we can't make $(y_i - \hat{y}_i) = 0$ if $(y_i - \hat{y}_i) = 0$ then $w_n = w_0$ so eqn line will never change.

$$w_n = w_0 + \eta (y_i - \hat{y}_i) X_i$$

$$(y_i - \hat{y}_i) \neq 0$$

The Sigmoid Function

$$\sigma = \frac{1}{1 + \exp(-z)}$$



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

if $\sigma(z)$ is +ve then $0.5 < \sigma(z) \leq 1$

if $\sigma(z)$ is -ve then $0 < \sigma(z) < 0.5$

$$m = -\frac{A}{B} \quad \text{coef}[0]$$

$$c = \frac{-C}{B} \quad \text{coef}[1]$$



$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i) - (1-y_i) \log(1-\hat{y}_i)$$

log loss error function / binary cross entropy

$$L = - \frac{1}{n} \sum_{i=1}^n y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)$$

Derivative of sigmoid:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\sigma'(x) = \frac{d}{dx} \left(\frac{1}{1+e^{-x}} \right)$$

$$\left[\frac{d}{dx} \left(\frac{1}{x} \right) = \frac{d}{dx} (x)^{-1} \right] \dots$$

$$\frac{d}{dx} \left[\frac{1}{1+e^{-x}} \right] = \frac{d}{dx} \left[(1+e^{-x})^{-1} \right]$$

$$= -1 \frac{d}{dx} (1+e^{-x}) \dots (\text{chain rule})$$

$$= -1 \frac{d}{dx} (e^{-x})$$

$$= -e^{-x} \frac{d}{dx} (-x) = \frac{e^{-x}}{(1+e^{-x})^2}$$

$$= \frac{1 \cdot e^{-x}}{(1+e^{-x})(1+e^{-x})} = \frac{1}{1+e^{-x}} \times \frac{e^{-x}}{1+e^{-x}}$$

$$= \sigma(x) \left[\frac{e^{-x}}{1+e^{-x}} \right] \dots \left[\frac{1}{1+e^{-x}} = \sigma(x) \right]$$

$$= \sigma(x) \left[\frac{1+e^{-x}-1}{1+e^{-x}} \right] = \sigma(x) \left[\frac{1+e^{-x}}{1+e^{-x}} - \frac{1}{1+e^{-x}} \right]$$

$$= \sigma(x) [1 - \sigma(x)] \Rightarrow$$

$$= \boxed{\sigma'(x) = \sigma(x) [1 - \sigma(x)]}$$

Gradient Descent for Logistic Reg.Rows = m Columns = n

1	2	3	...	n	y
x_{11}	x_{12}	x_{13}		x_{1n}	y_1
x_{21}	x_{22}	x_{23}		x_{2n}	y_2
x_{31}	x_{32}	x_{33}		x_{3n}	y_3
$\vdots$					$\vdots$
x_{m1}	x_{m2}	x_{m3}		x_{mn}	y_m

Coeff = $\{w_1, w_2, w_3, \dots, w_n\} \Rightarrow (n+1)$

$$\sigma(w_1 x_{11} + w_2 x_{12} + w_3 x_{13} + \dots + w_n x_{1n} + w_0) = y_1$$

$$\sigma(w_1 x_{21} + w_2 x_{22} + w_3 x_{23} + \dots + w_n x_{2n} + w_0) = y_2$$

$$\hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \sigma(w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n}) \\ \sigma(w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n}) \\ \vdots \\ \sigma(w_0 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn}) \end{bmatrix}$$

$$\hat{Y} = \sigma \begin{bmatrix} w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n} \\ w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n} \\ \vdots \\ w_0 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn} \end{bmatrix}$$

$$\hat{Y} = \sigma \left(\begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \right)$$

$$\boxed{\hat{Y} = \sigma(XW)}$$



Loss fun.

$$L = -\frac{1}{n} \sum_{i=1}^n y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)$$

$$L = -\frac{1}{n} \left[\underbrace{\sum_{i=1}^n y_i \log(\hat{y}_i)}_A + \underbrace{\sum_{i=1}^n (1-y_i) \log(1-\hat{y}_i)}_B \right]$$

take, A.

$$\sum_{i=1}^n y_i \log(\hat{y}_i) = y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2 + y_3 \log \hat{y}_3 + \dots + y_m \log \hat{y}_m$$

we also can write it as.

$$\begin{bmatrix} y_1 & y_2 & y_3 & \dots & y_m \end{bmatrix} \cdot \begin{bmatrix} \log \hat{y}_1 \\ \log \hat{y}_2 \\ \vdots \\ \log \hat{y}_m \end{bmatrix}$$

$$\begin{bmatrix} y_1 & y_2 & y_3 & \dots & y_m \end{bmatrix} \log \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \hat{y}$$

$$y \log \hat{y} = y \log(\sigma(xw)) \quad \hat{y} = \sigma(xw)$$

$$L = -\frac{1}{n} \left[y \log \hat{y} + (1-y) \log(1-\hat{y}) \right]$$

$$\text{where } \hat{y} = \sigma(xw)$$



Loss function in matrix form.

$$L = -\frac{1}{m} \left[Y \log(\sigma(WX)) + (1-Y) \log(1-\sigma(WX)) \right]$$

• Applying Gradient descent

$$\frac{dL}{dW} = -\frac{1}{m} \left[Y \log \hat{y} + (1-Y) \log(1-\hat{y}) \right]$$

$$\frac{d}{dW} Y \log \hat{y} \Rightarrow Y \frac{d}{dW} \log \hat{y} \Rightarrow Y \frac{d(\hat{y})}{\hat{y} \frac{dL}{dW}}$$

$$\Rightarrow \frac{Y}{\hat{y}} \frac{d}{dL} \sigma(WX) \Rightarrow \frac{Y}{\hat{y}} \sigma(WX) [1-\sigma(WX)] \frac{d(WX)}{dW}$$

$$\Rightarrow \frac{Y}{\hat{y}} \cdot \hat{y} (1-\hat{y}) X = \boxed{Y(1-\hat{y}) X}$$

$$\hat{y} = \sigma(WX)$$

$$\frac{d}{dW} (1-Y) \log(1-\hat{y}) \Rightarrow (1-Y) \frac{d}{dW} \log(1-\hat{y})$$

$$\Rightarrow \frac{(1-Y)}{(1-\hat{y})} \cdot \frac{d}{dW} [1-\hat{y}]$$

$$\Rightarrow -\frac{(1-Y)}{(1-\hat{y})} \cdot \frac{d}{dW} \sigma(WX)$$

$$\Rightarrow -\frac{(1-Y)}{(1-\hat{y})} \cdot \sigma(WX) [1-\sigma(WX)] \frac{d(WX)}{dL}$$

$$\Rightarrow -\frac{(1-Y)}{(1-\hat{y})} \cdot \hat{y} (1-\hat{y}) X$$

$$\Rightarrow \boxed{-\hat{y}(1-Y) X}$$

$$\frac{dL}{dW} = -\frac{1}{m} \left[Y(1-\hat{y}) X - \hat{y}(1-Y) X \right]$$



$$= -\frac{1}{m} [y(1-\hat{y}) - \hat{y}(1-y)] x$$

$$= -\frac{1}{m} [y - \cancel{y\hat{y}} - \hat{y} + \cancel{y\hat{y}}] x$$

$$\Delta L = -\frac{1}{m} (y - \hat{y}) x$$

$$\Delta W =$$

according to gradient descent

$$W = W + \eta \frac{1}{m} (y - \hat{y}) x$$

where

$$W = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix} \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

$(n+1, 1) \quad m, n+1 \quad (m, 1)$

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

$(m, 1)$

$$W = W + \boxed{\eta \frac{1}{m}} (y - \hat{y}) x$$

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$
 $(n+1, 1) \quad (n+1, 1) \quad (m, 1) \quad m, (n+1)$
 $(1, m+1)$

Accuracy and Confusion Matrix

23



SARASWATI Education Society's

PAGE NO.: 87

SARASWATI College of Engineering

DATE: 10/9/24

CLASSIFICATION METRICS

Accuracy

$$\text{Accuracy} = \frac{\text{No. of Correct Prediction}}{\text{Total No. of Predictions}}$$

implement

```
from sklearn.metrics import accuracy_score  
accuracy_score(y_true, y_pred)
```

• Problem with Accuracy:

Accuracy never tells the Precision & Recall Trade-off.

- Imbalanced datasets - accuracy is misleading with imbalanced dataset.
- Equal weightage for Errors. Accuracy treats all errors equally like False negative & False positive are not differentiable in accuracy score.

Confusion Matrix

		Prediction		P	
		1	0	1	0
Actual	1	True P Positive	False Negative	1 TP	0 FN
	0	False Positive	True Negative	0 FP	1 TN

1 = Yes

0 = No.

TP = True Positive

TN = True Negative

FP = False Positive

FN = False Negative



accuracy from confusion matrix

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} //$$

Type I error

FP is also known as Type I error.

Type II error

FN is known as Type II error.

implement.

```
from sklearn.metrics import confusion_matrix  
confusion_matrix(y_true, y_pred)
```

• Precision

Measures the proportion of true positive prediction among all positive

- Proportion of predicted positive
- Not More Harmful than Recall

$$\text{Precision} = \frac{TP}{TP + FP}$$

• Recall

Measures the proportion of correctly predicted positive observation to all observations in actual class.

Cost of FN is high

$$\text{Recall} = \frac{TP}{TP + FN}$$



F1 score

- It is used to penalize the lower value. It calculates the harmonic mean for predictions.

$$F1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

* Multiclass Classification *

Here we calculate all classes precision.

• Precision

① Macro precision: for balanced classes take avg of all classes precision

② Weighted Precision: for unbalanced / weighted biased classes.

all precision values are multiplied by their class weight

Class w.t. = $\frac{\text{each class actual value}}{\text{total items}}$

for 3 precisions.

$$\text{Macro} = \frac{P_1 + P_2 + P_3}{3}$$

$$\text{Weighted} = \text{Class wt}_1 \times P_1 + \text{Class wt}_2 \times P_2 + \text{Class wt}_3 \times P_3$$

• Recall

$$= \frac{\text{Total Predicted class value}}{\text{Total actual class value}}$$

— macro

— weighted



F1-Score

F1-Score is similar for all like
• We have to calculate F1-score individually for each class.

E.g. Precision = P_1, P_2, P_3

Recall = R_1, R_2, R_3

$$F1\text{-score}_1 = \frac{2 P_1 \times R_1}{P_1 + R_1}$$

$$F1\text{-score}_2 = \frac{2 P_2 \times R_2}{P_2 + R_2}$$

$$F1\text{-score}_3 = \frac{2 P_3 \times R_3}{P_3 + R_3}$$

implementation using sklearn

- Macro

```
from sklearn.metrics import precision_score,  
recall_score,  
f1_score
```

```
precision_score(y_test, y_pred, average='macro')
```

same for Recall & f1-score.

- Weighted

```
recall_score(y_test, y_pred, average='weighted')
```

same for precision-score & f1-score.

- All in One

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```



Softmax Regression

Softmax regression (or multinomial logistic regression) is a generalization of logistic regression to the case where we want to handle multiple classes.

Softmax regression allows us to handle $y(i) \in \{1, 2, \dots, K\}$ where K is the number of classes.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where, K is class
 $K = \{1, 2, \dots, K\}$

e.g. softmax fun apply for 1st class of 3 classes

if

$$\sigma(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$0 < \sigma < 1$$

(classes = 0, 1, 2 etc.)
Yes, No, Off etc.

Implementation

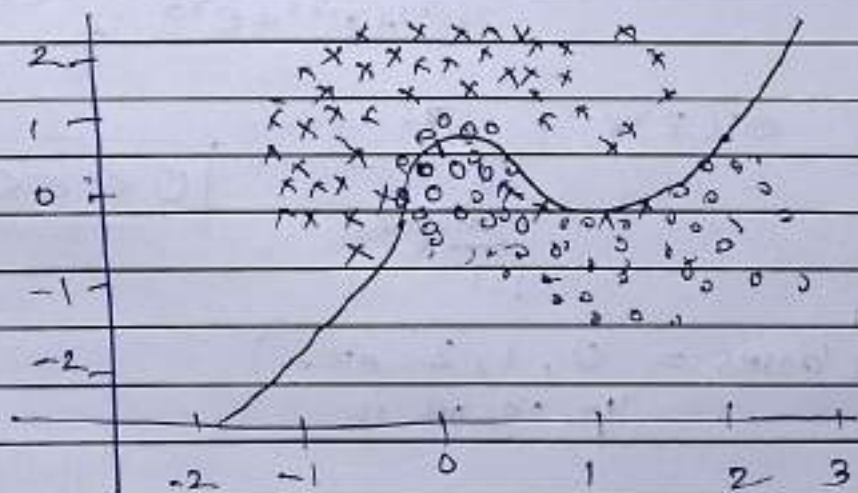
```
from sklearn.linear_model import LogisticRegression  
clf = LogisticRegression(multi_class='multinomial')  
clf.fit(X_train, y_train)  
clf.predict(X_test)
```



Polynomial Logistic Regression

Polynomial Features are created by transforming the original input features into a new set of feature that include original features & their polynomial combination up to specific degree. The allows transformation allows logistic regression to capture non-linear relationships between one input variables & the target variable.

Logistic regression with polynomial features is technique used for handling non-linear relationships in data. The idea is to transform input features into higher-degree polynomial, to capture more complex relationship between variables.



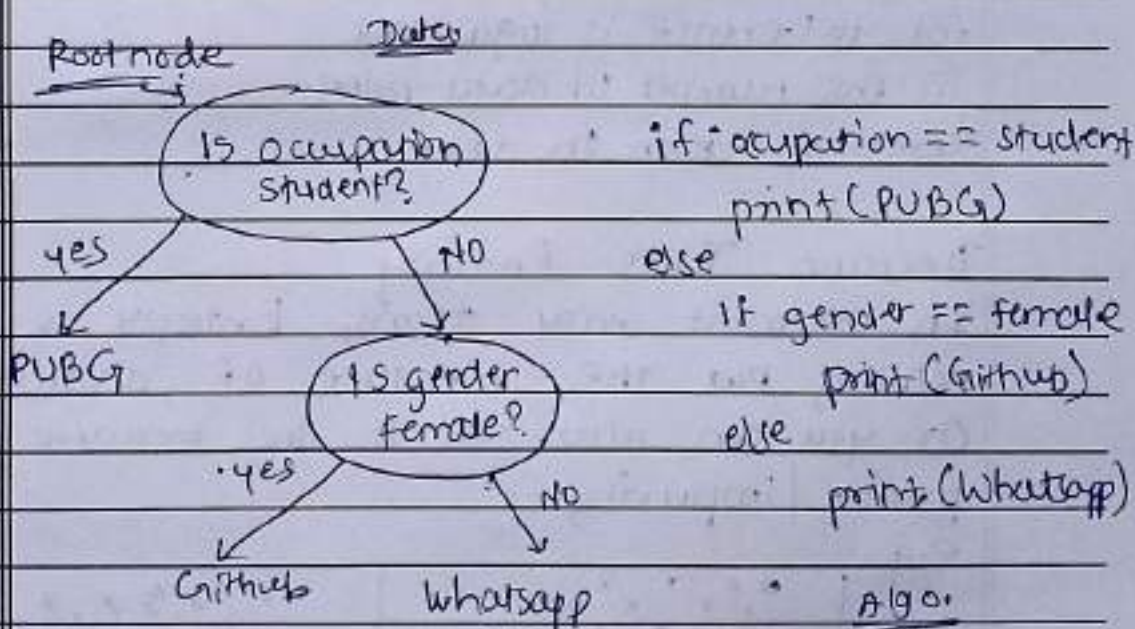


A decision tree is a decision support hierarchical model that uses a tree-like model of decision and their possible consequences, chance event outcomes, resource costs & utility. It is one way to display an algorithm that only contains conditional control statements.

Example

Gender Occupation Suggestion

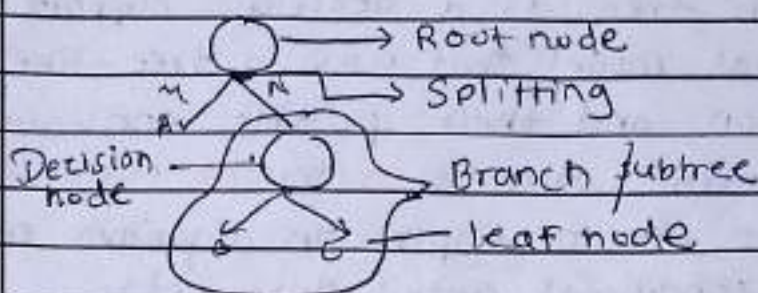
F	Student	PUBG
F	Programmer	GitHub
M	Programmer	Whatsapp
F	Programmer	GitHub
M	Student	PUBG
M	Student	PUBG



Tree



Terminology



Advantages

- Intuitive & easy to understand
- minimal data preparation is required

Disadvantages

- Overfitting
- prone to errors for imbalanced datasets

The cost of using one tree for inference is logarithmic in the number of data points used to train the tree

Decision Trees Entropy

- In the most easy terms, Entropy is nothing but the measure of disorder. Or you can also call it the measure of purity / impurity.
- e.g



High entropy



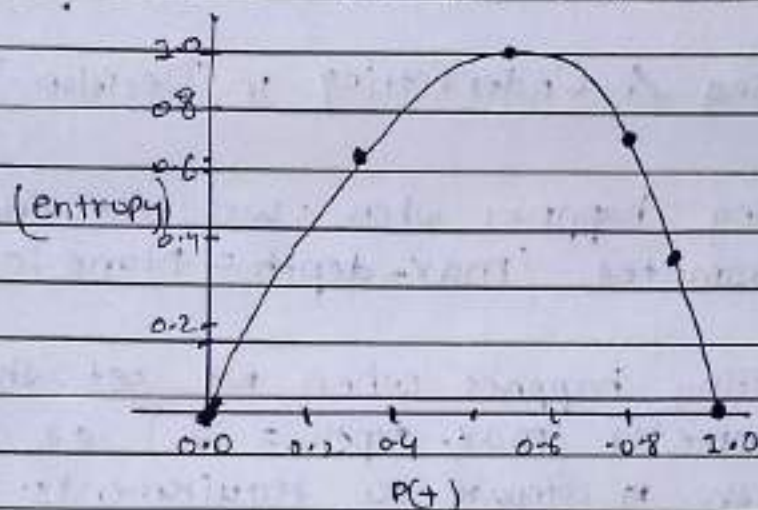
Low entropy



$$E(S) = \sum_{i=1}^n -P_i \log_2 P_i$$

where P_i frequentist probability of an element (class 'i' in our data)

$$E(D) = -P_{yes} \log_2 (P_{yes}) - P_{no} \log_2 (P_{no})$$



* Information Gain *

Information Gain, is a metric used to train Decision Trees. Specifically; this metric measures the quality of a split.

The information gain is based on the decrease in entropy after a data set is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain.

$$\text{Info Gain} = E(\text{parent}) - \{ \text{Weighted Average} \}$$
$$* E \{ \text{Children} \}$$



$$\text{Weighted Entropy} = P * E$$

* Gini Impurity *

$$G.I = 1 - (P_Y^2 + P_N^2)$$

* Overfitting & Underfitting in Decision Trees.

Overfitting happens when we set the hyperparameter (max-depth = None) or high

Underfitting happens when we set the hyperparameter (max-depth = 1) or very less with compare to requirements.

* Regression Trees *

Used when non-linear Relationship in dataset is present

For Decision Tree visualization

Use library called dtreeviz

install

pip install dtreeviz

import

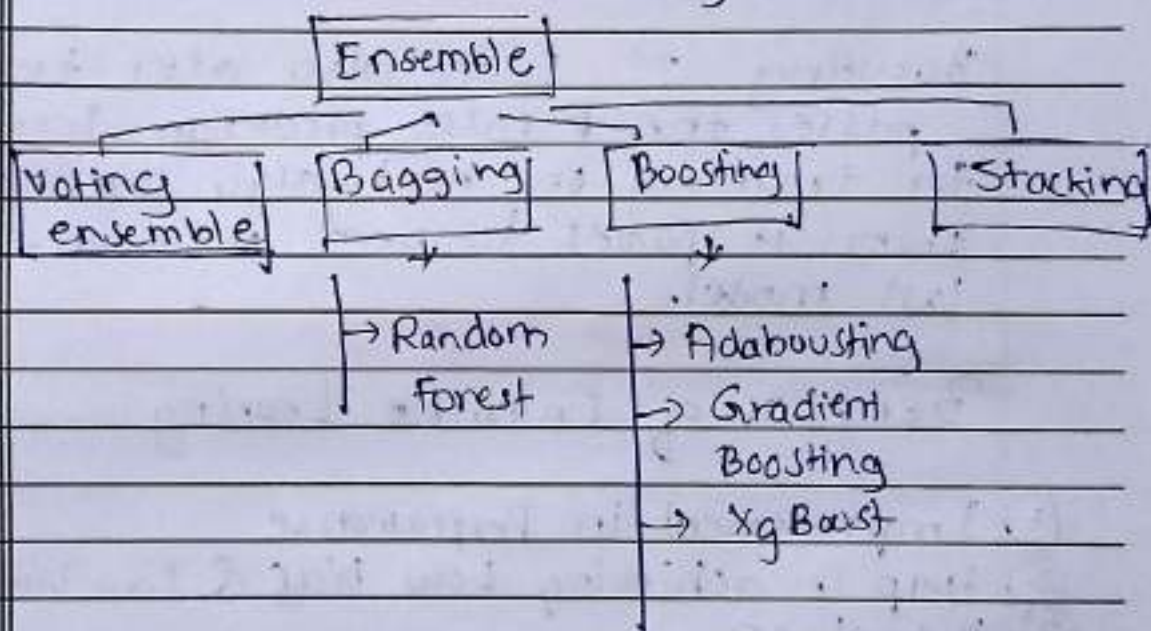
from dtreeviz.trees import *



Ensemble methods use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algo.

Ensemble learning trains two or more ml algorithms to a specific classification or regression task.

Type of Ensemble Learning



1. Voting Ensemble

- Base models are different Algo's and same Data.

2. Stacking

- Same as Voting but additionally we give weights to every models' output. and another best model is trained with results + additional weights of



remaining models.

Bagging $\rightarrow$ Bootstrapped Aggregation.
- Based models are same but the data is different for each model.

Random Forest $\rightarrow$ it is special case of bagging.

Boosting $\rightarrow$ Each algo after training passes the false prediction forward for training the next algo correctly then it passes to next on and on... till last model.

Benefits of Ensemble Learning

- ① Improvement in Performance
- ② Help in achieving Low bias & Low Variance
- ③ Robustness



Voting Ensemble

Ensemble voting is a popular technique in ensemble learning where the final prediction is made by combining the predictions of multiple individual models.

It can be used for classification & regression problems.

Voting Classifier

The concept is to build a single model that learns from various models and predicts output based on their aggregate majority and predicts output based on their of votes for each output class, rather than building separate specialized models & determining the accuracy for each of them.

2 different types of voting classifier

- Hard Voting: The predicted output class is a class with the highest majority of votes.

- Soft Voting: The average probabilities of the classes determine which one will be the final prediction.



To import voting.

```
>> from sklearn.ensemble import VotingClassifier
```

```
# Hard voting-
```

```
>> vc = VotingClassifier(estimators = [ ('lr', Logistic  
Regression()), ('rf', RandomForestClassifier()),  
( 'knn', KNeighborsClassifier(.) ),  
Voting = "hard" )
```

```
# Soft Voting
```

```
vc = VotingClassifier(estimators = estimators,  
Voting = "soft")
```

Voting regressor

It combines or 'ensembles' multiple regression models & overperforms the individual models present as its estimators. ml models as $m_1, m_2, m_3, \dots, m_n$. each model will produce a prediction P_n for given data I . If we pass through Voting Regressor then final prediction will be P_{voting} is Average of all predictions.

$$P_{\text{voting}} = \frac{1}{n} \sum_{n=1}^n P_n \Rightarrow \text{Simple avg}$$

$$P_{\text{voting}} = \sum_{n=1}^n (w_{tn} \cdot P_n) \Rightarrow \text{weighted avg}$$

where w_{tn} is custom w.t.

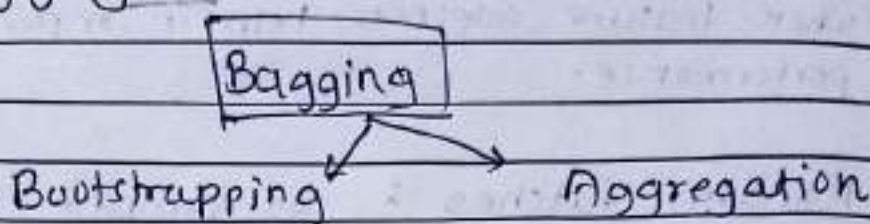
implementation

```
>> from sklearn.ensemble import VotingRegressor
```

```
>> vr = VotingRegressor(estimators)
```



Bagging



Bagging also known as bootstrap aggregation, is commonly used to reduce variance within a noisy dataset.

In bagging, a random sample of data in a training set is selected with replacement. After generating several data samples, these weak models are then trained independently. Depending on the type of task - regression or classification,

There are 3 different ways for performing Bagging

1. Pasting
2. Random Subspace
3. Random Patches

1. Pasting : It is sampling without replacement. Each subset is created by randomly sampling the training data without duplicating instances.

Use case: When dataset is large enough, & risk of overfitting due to duplicated instances in bootstrapped samples is a concern.

2. Random Subspace : It involves training each base model on a random subset of the features rather than the instances.



Use case - dealing with high dimensional data where feature selection help in improving model performance.

3. Random Patches : It combine both the random subspace & patching methods. It involves training each base model on a random subset of both instance & features.
Use case - beneficial when dataset is large & high-dimensional.

Bagging Classifier

Is an ensemble meta-estimator that fits base classifier each on random subsets of original dataset & then aggregate their individual predictions to form final prediction.

```
class sklearn.ensemble.BaggingClassifier (estimator=  
None, n-estimator=10, max-samples=1.0, max-  
features=1.0, bootstrap=True, bootstrap-features=  
False, oob-score=False, warm-start=False,  
n-jobs=None, random-state=None, verbose=0)
```

Implementation of code

```
>> from sklearn.ensemble import BaggingClassifier  
>> bag = BaggingClassifier (estimator = DecisionTree  
Classifier(), n-estimator = 500, max-samples=  
0.5, bootstrap = True, random-state = 42)  
>> bag.fit (X-train, y-train)  
>> bag.predict (X-test)
```



Bagging Regressor

A bagging regressor is an ensemble estimator that fits base regressors each on random subset of the original dataset & then aggregate their individual predictions (either by voting or averaging) to form a final prediction.

```
class sklearn.ensemble.BaggingRegressor (estimator=None,  
n_estimators=10, max_samples=1.0,  
max_features=1.0, bootstrap=True,  
bootstrap_features=False, oob_score=False,  
warm_start=False, n_jobs=None,  
random_state=None, verbose=0)
```

implementation of code

```
>> from sklearn.ensemble import BaggingRegressor  
>> bag = BaggingRegressor (random_state=1)  
>> bag.fit(X_train, y_train)  
>> bag.predict(X_test)
```



Random forests or random decision forests is an ensemble method for classification, regression and other task that operates by constructing a multitude of decision trees at training time.

For classification tasks, the ~~mean~~ output of the random forest is the class selected by most trees.

For regression tasks, the mean or average prediction of the individual trees is returned.

Random Forests converts the ~~High Bias - High Variance~~ HBHV model.

Low Bias High Variance algorithm into
Low Bias Low Variance algorithm

LBHV converts LBLV

Difference Between: Bagging & Random Forest
Bagging-

- ① In Bagging we can use multiple algorithms (Decision Tree, SVM, KNN, etc.) it is generalize method.
- ② In Bagging Feature / column sampling the No. of columns choosed in starting are always fixed for all trees creation.
- ③ Column Sampling is at tree level.
Columns not shuffle they are fixed.
- ④ eg if 2 columns are selected col 3, & col 5
So in all model it will be fixed.



Random Forest

- ① In Random Forest we only use Decision Tree model / Algorithm. It is focused method.
- ② Column Sampling at node level
- ③ Column / feature Sampling the no. of columns choose in starting is fixed and the internal columns are shuffle in each node.
eg. if 2 columns are selected. col 2, & col 4
after 1 node again randomly. columns will shuffle now it will choose col 3, col 1
for next node it will go on & on..

Random Forest Classifier.

```
class sklearn.ensemble.RandomForestClassifier (n_estimators=100,  
criterion = 'gini', max_depth = None, min_samples_split=2,  
min_samples_leaf=1, max_features = 'sqrt', max_leaf_nodes = None,  
min_impurity_decrease = 0.0, bootstrap=True,  
oob_score = False, n_jobs = None, random_state=None,  
verbose=0, warm_start=False, ccp_alpha=0.0,  
class_weight = None, max_samples = None,  
monotonic_cst = None)
```

implementation in code

```
>> from sklearn.ensemble import RandomForestClassifier  
rf = RandomForestClassifier (n_estimators=500,  
random_state = 42, max_features=2)  
rf.fit (X_train, y_train)  
rf.predict (X_test)
```



RandomForestRegressor

is a meta estimator that fits a no. of decision tree regressors on various sub-samples of the dataset & uses averaging to improve the predictive accuracy.

Syntax.

```
class sklearn.ensemble.RandomForestRegressor (n_estimators=100, *, criterion='squared_error', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=1.0, max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, ccp_alpha=0.0, max_samples=None, monotonic_cst=None)
```

implementation using sklearn..

```
>> from sklearn.ensemble import RandomForestRegressor
>> rfr = RandomForestRegressor(n_estimators=1000)
rfr.fit(x_train, y_train)
rfr.predict(x_test)
```



* Out of Bag Evaluation OOB - Score

Random forests do not require validation dataset. Most random forests use a technique called out-of-bag-evaluation (OOB evaluation) to evaluate the quality of the model. OOB evaluation treats the training set as if it were on the test set of a cross-validation.

Over $\sim 33\%$ of training examples are missed by decision tree while choosing data with replacement.

Implementation:

```
from sklearn.ensemble import RandomForestClassifier  
rfc = RandomForestClassifier(oob_score=True)  
rfc.fit(X_train, Y_train)  
rfc.predict()  
rfc.oob_score_
```

* Feature Importance using Random Forest & Decision Trees

It also known as variables or attributes, are individual measurable properties or characteristics of the phenomena being observed.

Formula for Decision trees feature importance.

$$f_{ik} = \frac{\sum_{j \in \text{node split on feature } k} n_i}{\sum_{j \in \text{all nodes}} n_i}$$



$$n_i = \frac{N-t}{N} \left[\text{impurity} - \left(\frac{N-t-r}{N-t} * \text{right-impurity} \right) - \left(\frac{N-t-l}{N-t} * \text{left-impurity} \right) \right]$$

where, n_i = node importance

$N-t$ = No. of rows to Node

N = Total No. of rows present in data

impurity = 'gini'

$N-t-r$ = Node to right

$N-t-l$ = Node to left.

right-impurity = impurity on right node

left-impurity = impurity on left node

$$\leq n_i$$

$$f_{ik} = \frac{j \in \text{nod split on feature } k}{\sum_{j \in \text{all nodes}} n_j}$$

Implementation of Random forest feature importance

```
>> from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(X, y)
rf.feature_importances_
```

```
>> from sklearn.ensemble import DecisionTreeClassifier
clf = DecisionTreeClassifier()
clf.fit(X, y)
clf.feature_importance_
```



Implementation of Decision Tree importance



ADABOOST

AdaBoost Adaptive Boosting is used in conjunction with many other algos to improve performance. The output of the other learning algorithms ('Weak learners') is combined into a weighted sum that represents the final output of the boosted classifier.

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{error}}{\text{error}} \right)$$

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - E_m}{E_m} \right)$$

where E_m = weighted error rate of the weak classifier.

α_m = weight of model m .

$$E_m = \frac{\sum_{i=1}^N \mathbb{I}_{y_i \neq k_m}(x_i) w_i^{(m)}}{\sum_{i=1}^N w_i^{(m)}}$$

For misclassified

$$\text{new_wt} = \text{curr_wt} * e^{\alpha}$$

For correctly classified

$$\text{new_wt} = \text{curr_wt} * e^{-\alpha}$$



AdaBoost Classifier

An AdaBoost classifier is a meta-estimator that begins by fitting a classifier on one original data set, then fits

Syntax

```
class sklearn.ensemble.AdaBoostClassifier( estimator=  
None, *, n_estimators = 50, learning_rate = 1.0,  
algorithm = 'SAMME.R', random_state = None)
```

Implementation in sklearn.

```
>> from sklearn.ensemble import AdaBoostClassifier  
abc = AdaBoostClassifier()  
abc.fit(X, y)  
abc.predict(x-test)
```

Bagging

Vs Boosting

①	Types of model used <u>LBHV</u> model used	HBLV model used
②	Full grown decision tree	Shallow grown decision tree
③	Parallel training	Sequential (one-by-one)
④	equally weightage of base learners	biased or unequal weightage of base learners.

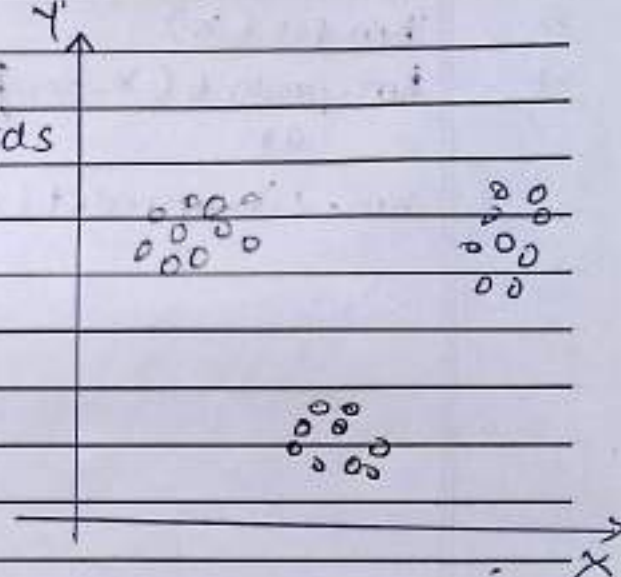


K-means clustering

Is a unsupervised learning method of vector quantization, that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or centroid), serving as a prototype of the cluster.

Steps for achieving k-means cluster.

- 1) Decide n clusters
- 2) Initialize n centroids
- 3) Assign cluster
- 4) Move Centroid
- 5) Finish



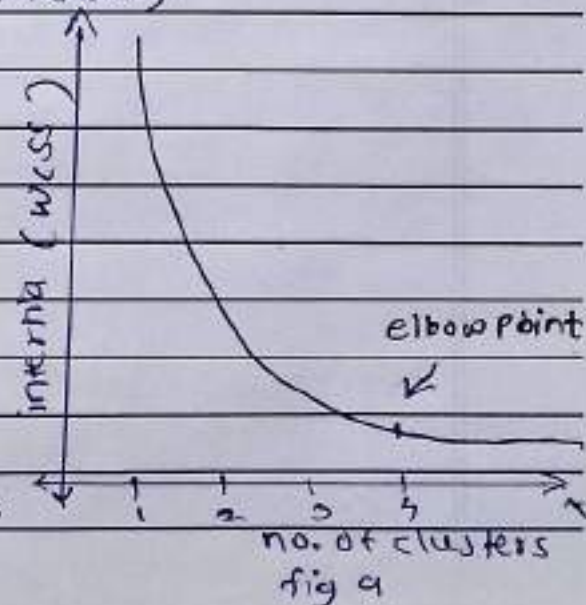
To find or Decide number of clusters we follow (Elbow Method)

= WCSS =) within clusters sum of squared distance

$$WCSS = \sum_{i=1}^n d_i^2$$

A one point where WCSS got stable we choose no. of cluster

in fig 4 clusters is chosen





Syntax.

```
class sklearn.cluster.KMeans(n_clusters = 8, *,  
    init = 'k-means++', n_init = 'auto', max_iter = 300,  
    tol = 0.0001, verbose = 0, random_state = None,  
    copy_x = True, algorithm = 'lloyd')
```

implementation in sklearn

```
>> from sklearn.cluster import KMeans  
>> km = KMeans(n_clusters = 4)  
>> km.fit(X)  
>> km.predict(X_new) X  
    or  
km.fit_predict(X)
```



Gradient boosting is a ML technique based on boosting in a functional space, where the target is pseudo-residuals rather than the typical residuals used in traditional boosting. It gives a prediction model in the form of an ensemble of weak prediction models.

Gradient boosting is used with decision trees of a fixed size as base learners.

* Algorithm *

Input: training set $\{(x_i, y_i)\}_{i=1}^n$, a diff. loss function $L(y, f(x))$, no of iterations M .

1. Initialize $f_0(x) = \arg \min_f \sum_{i=1}^n L(y_i, f)$.

2. For $m=1$ to M :

(a) For $i=1, 2, \dots, N$ compute

$$\gamma_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}$$

(b) Fit a regression tree to the targets γ_{im} giving terminal regions R_{jm} , $j=1, 2, \dots, J_m$.

(c) For $j=1, 2, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

(d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.



The Loss function that we have used in the above algo is Least square.

$$L = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

(MSE)

Syntax for Regressor.

```
class sklearn.ensemble.GradientBoostingRegressor(  
    *, loss = 'squared-error', Learning_rate = 0.1,  
    n_estimators = 100, subsample = 1.0, criterion = 'friedman-  
    mse', minsample-split = 2, min-sample-leaf = 1,  
    min-weight-fraction-leaf = 0.0, max-depth = 3,  
    min-impurity-decrease = 0.0, init = None, Random_state = None,  
    max-features = None, alpha = 0.9, verbose = 0,  
    max-leaf-nodes = None, warm-start = False,  
    validation-fraction = 0.1, n_iter_no_change = None,  
    tol = 0.0001, ccp-alpha = 0.0)
```

Implementation in sklearn.

```
from sklearn.ensemble import GradientBoostingRegressor  
reg = GradientBoostingRegressor(random_state=0)  
reg.fit(X_train, y_train)  
reg.predict(X_test)
```



Syntax for Classifier.

```
class sklearn.ensemble.GradientBoostingClassifier(+,  
loss = 'log-loss', learning_rate = 0.1, n_estimators = 100,  
subsample = 1.0, criterion = 'friedman-mse',  
min_samples_split = 2, min_sample_leaf = 1,  
min_weight_fraction_leaf = 0.0, max_depth = 3,  
min_impurity_decrease = 0.0, init = None, random_state =  
None, max_features = None, verbose = 0, max_leaf_  
nodes = None, warm_start = False, validation_fraction =  
0.1, n_iter_no_change = None, tol = 0.0001, ccp_alpha = 0.0)
```

implementation

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
clf = GradientBoostingClassifier(n_estimators = 100,  
learning_rate = 1.0, max_depth =  
1, random_state = 0)
```

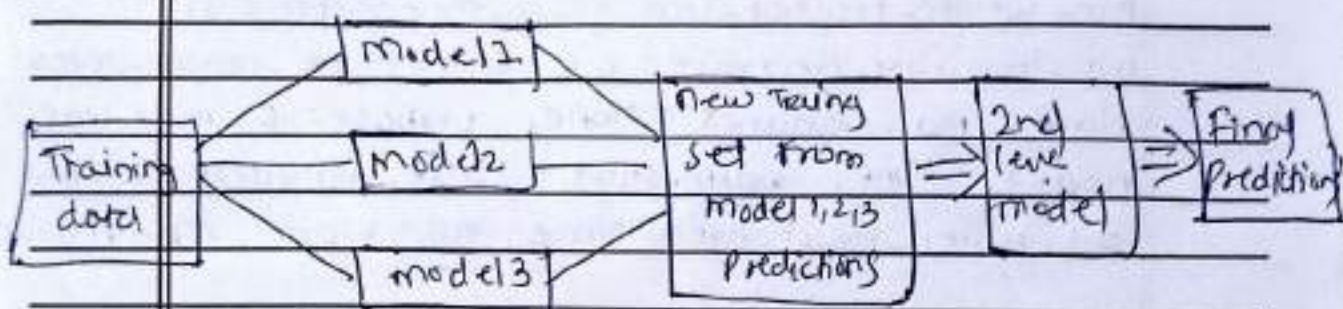
```
clf.fit(x_train, y_train)
```

```
clf.predict(x_test)
```



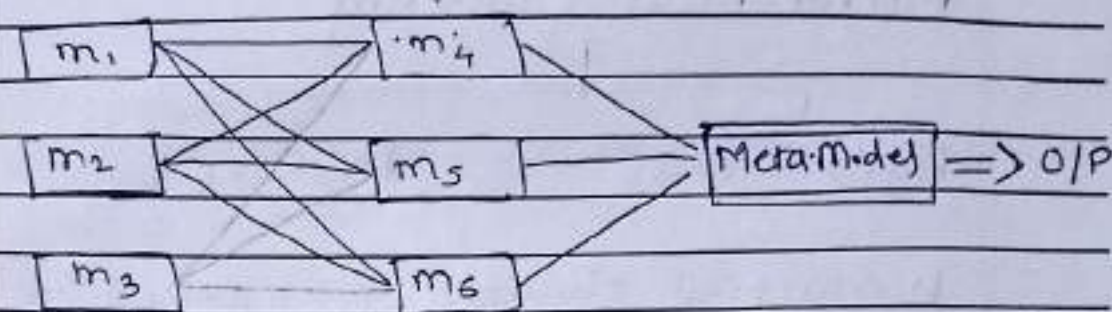
Stacking

Stacking is a way to ensemble multiple classifications or regression model. The point of stacking is to ~~ex~~ explore a space of different models for the same problem.



There are two or three methods for stacking.

- ① **Stacked Model** : An ensemble of models in which the meta-model is trained on out-of-fold predictions made by one base models during k-fold cross validation.
- ② **Blended Model** : An ensemble of models in which the meta-model is trained on predictions made by the base models on a holdout dataset (e.g. the validation dataset).
- ③ **Multi Layer Stacking** : This create a hierarchical structure of models where each layer's models are trained on predictions of the previous layer.



Syntax

```
class sklearn.ensemble.StackingClassifier(estimators,  
final_estimator=None, *, cv=None, stack_method=  
'auto', n_jobs=None, passthrough=False,  
verbose=0)
```

implementation using sklearn

```
from sklearn.ensemble import StackingClassifier  
estimators = [ ('rf', RandomForestClassifier(n_estimators=  
10, random_state=42)),  
               ('knn', KNeighborsClassifier(n_neighbors=10)),  
               ('gbdt', GradientBoostingClassifier()) ]  
from sklearn.linear_model import LogisticRegression  
clf = StackingClassifier(estimators=estimators,  
                          final_estimator=LogisticRegression(),  
                          cv=10)  
  
clf.fit(X_train, y_train)  
clf.predict(X_test)
```



Hierarchical Clustering

Agglomerative

Divisive

Hierarchical clustering is a method of cluster analysis that seeks to build a hierarchy of clusters. Strategies are of 2 ways.

- **Agglomerative clustering:** This is a "bottom-up" approach: Each obs. starts in its own cluster, & pairs of clusters are merged as one moves up the hierarchy.
- **Divisive clustering:** This is "top-down" approach: All observations start in one cluster, & splits are performed recursively as one moves down the hierarchy.

The merges & splits are determined in a greedy manner. The results of hierarchical clustering are usually presented in a dendrogram dendrogram.

Algorithm:

1. Initialize the Proximity Matrix
2. Make each point a cluster
3. Inside a loop
 - a. Merge the 2 closest clusters
 - b. Update the Proximity Matrix
4. Until only one cluster is left.



Types of Agglomerative Clustering

1. Min (single-link) $\Rightarrow$ 'single'
'single' uses the minimum of distances betⁿ all observations of the two sets.
2. Max (complete link) $\Rightarrow$ 'complete' or 'maximum'
uses the maximum distance betⁿ all observations of the two sets.
3. Average $\Rightarrow$ 'average'
uses the average of the distance of each obs of the two sets.
4. 'Ward'
minimizes the variance of clusters being merged

Syntax

```
class sklearn.cluster.AgglomerativeClustering (n_clusters  
= 2, *, metric = 'euclidean', memory = None, connectivity =  
None, compute_full_tree = 'auto', linkage = 'ward',  
distance_threshold = None, compute_distances = False)
```

Implementation

```
from sklearn.cluster import AgglomerativeClustering  
agc = AgglomerativeClustering()  
agc.fit(X)  
agc.labels_
```



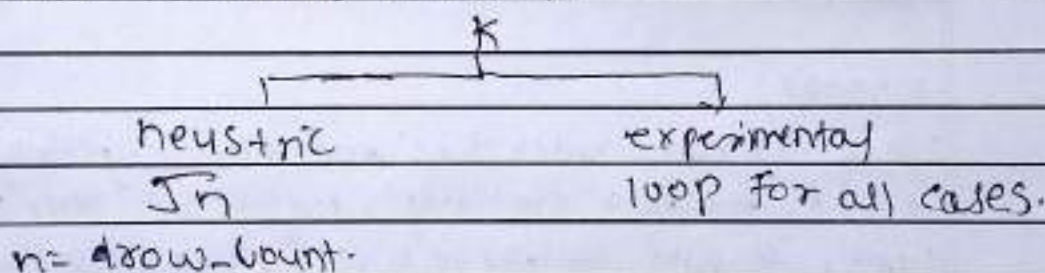
K-nearest neighbors algorithm (K-NN) is a non-parametric supervised learning method.

It is used for classification & regression.

K-NN algorithm assumes the similarity between the new case/data & available cases & put the new case into the category that is most-similar to the available categories.

K-NN alg. stores all the available data & classifies a new data point based on the similarity means when new data appears then it can be easily classified into a well suite category by using K-NN algorithm.

* How to select data K.



* Overfitting & Underfitting.

if $k=1$ or very low then $\Rightarrow$ Overfitting

if $k=n$ or near to n then $\Rightarrow$ Underfitting

$n = \text{no. of rows.}$

* Limitations

- 1) Large dataset $\Rightarrow$ Lazy learning technique
- 2) High dim data $\Rightarrow$ Curse of dimensionality.
- 3) Outliers



- a) Non-homogeneous Scales. (feature scales)
- s) Imbalanced dataset
- s) Inference and not for prediction. uses.

Syntax

```
class sklearn.neighbors.KNeighborsClassifier(  
    n_neighbors = 5, *, weights = 'uniform', algorithm =  
    'auto', leaf_size = 30, p = 2, metric = 'minkowski',  
    metric_params = None, n_jobs = None)
```

Implementation

```
from sklearn.neighbors import KNeighborsClassifier  
neigh = KNeighborsClassifier(n_neighbors = 3)  
neigh.fit(X, y)  
neigh.predict(X_test)
```



Assumptions of Linear Regression

1. Linear Relationship between input & output
2. No Multicollinearity
3. Normality of Residual
4. Homoscedasticity
5. No Autocorrelation of Errors

1 → Find linearity betⁿ each feature with target column.

2 → All input features should independent in nature

- calculate variance inflation factor for checking multicollinearity

- calculate correlation betⁿ columns for checking multicollinearity

3 → Error should be normally distributed

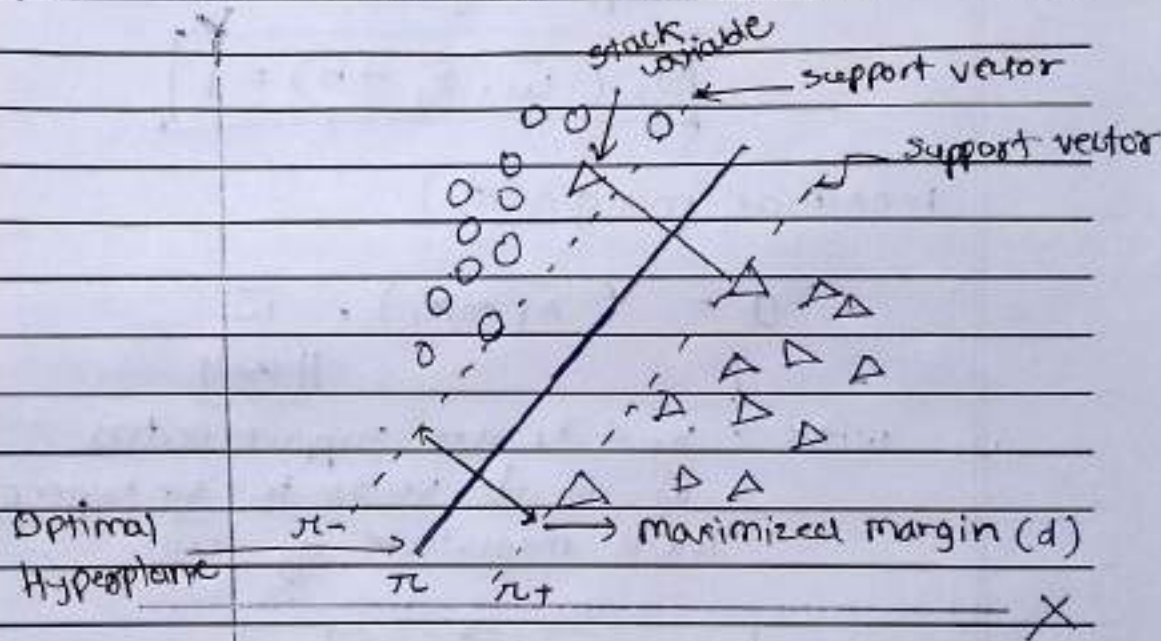
4 → Having the same scatter the ~~feature~~ spread should uniform on residual values.

5 → Should NO positive auto-correlation of Residuals.



Support Vector machines (SVM) are supervised max-margin models with associated learning algorithms that analyze data for classification & regression analysis.

SVM distinguish between two classes by finding the optimal hyperplane that maximizes the margin between the closest data points of opposite classes. SVM algorithm can handle both linear & nonlinear classification tasks.



Let's derive decision rule of SVM.
we know.

$$\vec{w} \cdot \vec{u}^+ > c$$

$$\vec{w} \cdot \vec{u}^- - c < 0$$

$$\vec{w} \cdot \vec{u} + b > 0$$

for any x_i

$$\hat{y} = \begin{cases} +1 & \text{if } \vec{w} \cdot \vec{x}_i + b \geq 0 \\ -1 & \text{if } \vec{w} \cdot \vec{x}_i + b < 0 \end{cases}$$

$\hat{y}$ = Predicted value.



assume $w^T x + b = 1$ for π +ve
 $w^T x + b = -1$ for π -ve

Optimization function

$$\begin{cases} \vec{w} \cdot \vec{x} + b \geq 1 \\ \vec{w} \cdot \vec{x} + b \leq -1 \end{cases} \quad \begin{cases} y_i (\vec{w} \cdot \vec{x}_i + b) \geq 1 \\ y_i (\vec{w} \cdot \vec{x}_i + b) \leq -1 \end{cases}$$

↖

$$y_i (\vec{w} \cdot \vec{x}_i + b) \geq 1$$

for support vectors.

$$y_i (\vec{w} \cdot \vec{x}_i + b) = 1$$

maximize margin (d)

$$d = \frac{(\vec{x}_2 - \vec{x}_1) \cdot \vec{w}}{\|\vec{w}\|}$$

where $\vec{x}_1$ & $\vec{x}_2$ are support vectors

$\vec{w}$ is $\perp$ vector to the hyperplane.

$\|\vec{w}\|$ magnitude of $\vec{w}$.

$$d = \frac{\vec{x}_2 \cdot \vec{w} - \vec{x}_1 \cdot \vec{w}}{\|\vec{w}\|} \quad \dots \dots \textcircled{6}$$

$$y_i (\vec{w} \cdot \vec{x}_i + b) = 1$$

$$(\vec{w} \cdot \vec{x}_2 + b) = 1$$

$$\vec{w} \cdot \vec{x}_2 = (1 - b) \quad \dots \dots \textcircled{7}$$

Similarly of x_1

$$-1 (\vec{w} \cdot \vec{x}_1 + b) = 1$$

$$\vec{w} \cdot \vec{x}_1 = -b - 1 \quad \dots \dots \textcircled{8}$$

replace the value in eqⁿ $\dots \textcircled{6}$



$$\frac{\vec{x}_2 \cdot \vec{w} - \vec{x}_1 \cdot \vec{w}}{\|\vec{w}\|} = \frac{(-b) - (-b-1)}{\|\vec{w}\|}$$

$$= \frac{1 - b + b + 1}{\|\vec{w}\|}$$

$$\boxed{d = \frac{2}{\|\vec{w}\|}} \quad \dots \quad (3)$$

to maximize the eqⁿ (3).

$$\begin{array}{l} \text{argmax} \quad \frac{2}{\|\vec{w}\|} \quad \text{such that} \\ (\vec{w}^*, b^*) \quad y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 \\ \quad \quad \quad y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 \end{array}$$

Hard Margin optimization function.

Hard Margin SVM: Is a type of SVM that tries to find a linear separator (hyperplane) that perfectly separates the data points of different classes without any errors.

Constraint: requires data to be perfectly separable.

Soft Margin SVM: It allows some data points to be misclassified or to be lie within the margin. it introduces a penalty for misclassified points to handle cases where data is not perfectly separable.

Constraint: It uses a regularization parameter (C) to control the trade-off between



achieving a larger margin and having a lower error rate. A smaller C allows more misclassifications, while a larger C tries to classify all training examples correctly.

The optimize function for soft margin is

$$\max f(w) \Leftrightarrow \min \frac{1}{f(w)}$$

$$\operatorname{argmin}_{(w^*, b^*)} \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i$$

subject to:

$$y_i (w \cdot x_i + b) \geq 1 - \xi_i \quad \text{for all } i = 1, 2, \dots, N$$

$$\xi_i \geq 0 \quad \text{for all } i = 1, 2, \dots, N.$$

where,

w is weight vector

b is bias term

(zeta) $\xi_i(x_i)$ are slack variables that allow some misclassification

C is regularization parameter Controls trade of betⁿ max one margin & min one misclassification error.

y_i is class label of i th e.g.

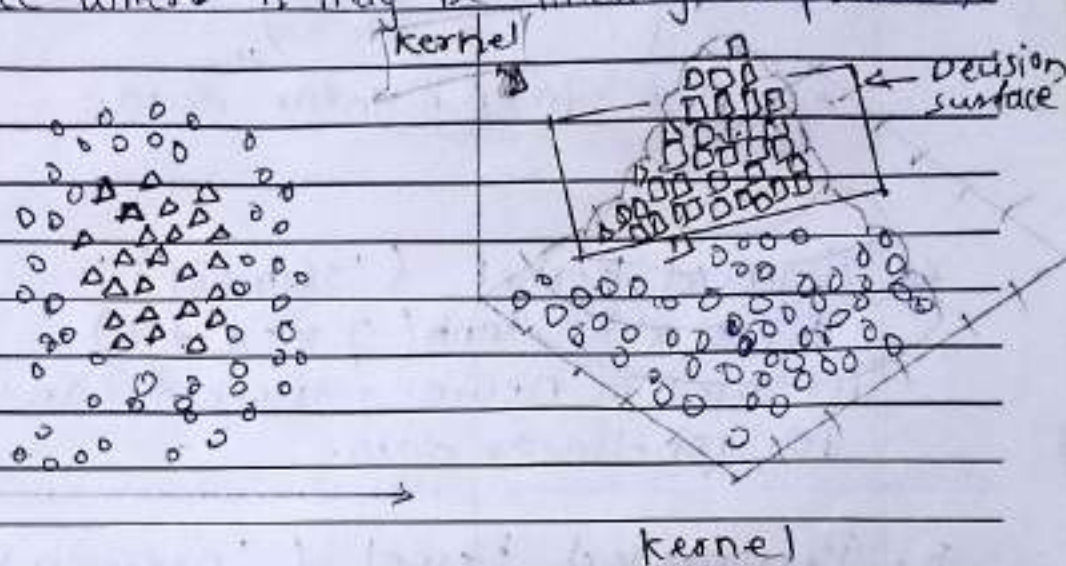
x_i is feature vector of i th training e.g.

N is no. of training. e.g.



Kernel Trick in SVM

The "Kernel Trick" is a method used in SVMs to convert data (that is not linearly separable) into a higher-dimensional feature space where it may be linearly separated.



different types of kernels used in SVM.

1. Linear kernel ('linear')

- It is dot product of two vectors used for linearly separable data. $K(x, x') = x \cdot x'$

2. Polynomial kernel ('poly'):

- is a non-linear kernel that represents the similarity of vectors in feature space over polynomial of original variables $K(x, x') = (\gamma x \cdot x' + r)^d$
 γ is scaling parameter, r is constant, d is degree of polynomial.

γ is scaling parameter

it is useful for scenarios where the relationship betⁿ class labels & attributes is non-linear.



3. Radial Basis Function (RBF) kernel ('rbf'):

- formula $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

$$\gamma = \frac{1}{2\sigma^2}$$

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

effective in non-linear data.

4. Sigmoid kernel ('sigmoid'):

$$K(x, x') = \tanh(\gamma x \cdot x' + c)$$

its used in neural network & can be used for non-linear data.

5. Precomputed kernel ('precomputed'):

- requires manually compute the kernel matrix before training SVM.

- kernel matrix should be an $n \times n$ matrix where n is no. of samples.

6. Callable kernel

custom kernel function as a callable:

fun should take arguments X & Y , & return kernel matrix.

e.g.

```
def custom_kernel(X, Y):  
    return np.dot(X, Y.T)
```



SVM syntax & parameters.

```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf',  
degree=3, gamma='scale', coef0=0.0,  
shrinking=True, probability=False, tol=0.001,  
cache-size=200, class-weight=None, verbose=False,  
max_iter=1, decision-function-shape='ovr',  
break-ties=False, random-state=None)
```

implementation.

```
>> from sklearn.svm import SVC  
>> model = SVC(kernel='linear', C=1)  
>> model.fit(X, y) # X=train, y=train  
>> model.predict(X-test)
```



NAIVE BAYES CLASSIFIER

SARASWATI Education Society's

PAGE NO.: 130

SARASWATI College of Engineering

DATE: 21/7/2024

Condition Probability

A, B:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \{P(B) \neq 0\}$$

Independent Events

A, B

$$P(A \cap B) = P(A) * P(B)$$

$$P(A|B) = \frac{n(A \cap B)}{n(B)}$$

$$P(A) = \frac{n(A \cap B)}{n(B)}$$

$$P(A|B) = P(A)$$

Mutually Exclusive Event

A, B

$$P(A \cap B) = 0$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(A|B) = 0$$

Bayes Theorem

We know, Conditional probability

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{--- (i)}$$

$$A \cap B = B \cap A$$

$$\therefore P(B|A) = \frac{P(B \cap A)}{P(A)} = \frac{P(A \cap B)}{P(A)} \quad \text{--- (ii)}$$

$$P(A \cap B) = P(B|A) * P(A) \quad \text{--- (ii)}$$

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)} \quad \text{--- (iii)}$$

↑
↓
prior
Evidence

Posterior

given $\{P(B) \neq 0\}$



Naive Bayes Classifier

Naive Bayes classifiers are a family of linear "probabilistic classifiers" which assumes that the features are conditionally independent, given the target class.

Naive Bayes is a conditional probability model it assigns probabilities $p(C_k | x_1, \dots, x_n)$ for each of the K possible outcomes or classes C_k given a problem instance to be classified, represented by a vector $x = (x_1, \dots, x_n)$ encoding n features.

$$P(C_k | x) = \frac{P(C_k) P(x | C_k)}{P(x)}$$

$$\text{Posterior} = \frac{\text{Prior} \times \text{likelihood}}{\text{evidence}} //$$

Types of Naive Bayes Classifiers

- Gaussian Naive Bayes (GaussianNB): This is used with Gaussian distributions - i.e. normal distributions - & continuous variables. This model is fitted by finding the mean and standard deviation of each class.

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2}$$

- Multinomial Naive Bayes (MultinomialNB): This type assumes that features are from multinomial distributions. This variant is useful when using discrete data, such as frequency.



counts, & it is typically applied within NLP.

- Bernoulli Naive Bayes (BernoulliNB): This classifier is used with Boolean variables. That is, variables with two values, i.e. True & False or 1 & 0.

Implementation.

```
from sklearn.naive-bayes import GaussianNB  
gnb = GaussianNB()  
gnb.fit(X_train, y_train)  
gnb.predict(X_test)
```



XGBoost (eXtreme Gradient Boosting)

is an open-source software library which provide a regularizing gradient boosting framework for C++, Java, Python, R, Julia, Perl, Scala. It works on Linux, Microsoft Windows & macOS. It runs on a single machine, as well as the distributed processing frameworks Apache Hadoop, Apache Spark, Apache Flink & Dask.

XGBoost initially started by Tiangi Chen as a part of Distributed ML community group (DMLC).

Developed by Tiangi Chen from University of Washington.

Features.

Performance Speed Flexibility

1. Flexibility

- Cross Platform support
- Multiple Language Support
- Integration with other libraries & tools
- Support all kinds of ML problems.



2. Speed

- Uses Parallel Processing in tree building stage
- Uses Optimized Data Structures
- Cache Awareness
- Out of Core Computing - sequential
- Distributed Computing - parallel
- GPU Support

3. Performance

- Regularized Learning Objective
- Handling Missing Values
- Sparsity Aware Split Finding
- Efficient Split Finding

(Weighted Quantile Sketch

+ Approximate Tree Learning)

(Convert Continuous var into discrete) uses histogram based training using weighted quantile sketch.

- Tree Pruning

Comparing With Other Boosting Algorithms

XGBoost vs. AdaBoost

AdaBoost Emphasizes incorrect predictions using weights. Correct predictions reduce weight, incorrect prediction increase weight.
less effective than XGBoost in accuracy & speed.



XGBoost Vs. CatBoost

CatBoost is another gradient boosting framework. It specializes in handling categorical features without preprocessing. perform well out-of-the-box with less hyperparameter tuning.

Like XGBoost, CatBoost has built in support for handling missing data.

CatBoost especially useful for datasets with many categorical features.

XGBoost vs. LightGBM

LightGBM (Light Gradient Boosting Machine)

LightGBM was developed by Microsoft in 2016

Most decision tree learning algorithm grow trees depth-wise. It uses ~~and~~ leaf-wise tree growth strategy.

Like XGBoost, LightGBM exhibits fast model training speed & accuracy & performance well with large datasets.



XGBoost for Regression

For split finding in Regression 2 algorithms are used.

- ① Exact Greedy Algorithm for Split finding
- ② Split finding
- ③ Approximate Algorithm for Split finding

XGBoost Objective function contains loss function & regularization term. it tells diff. betⁿ actual value & predicted values.

XGBoost expects to have base learners which are uniformly bad at one remainder so that when all the predictions are combined, bad predictions cancels out & better one sums up to form final good predictions. 1)

For building XGBoost tree for Regression:
step 1: Calculate the similarity scores. it helps in growing the tree.

$$\text{Similarity Score} = \frac{(\text{sum of residuals})^2}{\text{No. of residuals} + 2}$$

step 2: Calculate the gain to determine how to split the data.

$$\text{Gain} = \text{Left tree (Similarity score)} + \text{Right tree (Similarity score)} - \text{Root (Similarity score)}$$

$$[L + R - \text{Root}]$$



step 3: Prune the tree by calculating the difference between Gain & gamma
 $\boxed{\text{Gain} - \text{gamma}}$

if result is positive no. the do not prune
if result is negative, then prune & again subtract gamma from the next Gain value way up the tree.

step 4: Calculate output value for the remaining leaves

$$\text{Output value} = \frac{\text{Sum of residuals}}{\text{No. of residuals} + \lambda}$$

1st model is Avg of all x_i (feature values)
(base model)

$$\text{Similarity Score} = \frac{(\sum \text{Residuals})^2}{N + \lambda} //$$

$$\text{Output Value} = \frac{\sum \text{Residuals}}{N + \lambda} //$$

where N is no of residuals.



Loss Function / objective function.

Loss fun

regularization Para

$$L = \sum_{i=1}^n L(y_i, \hat{y}_i) + \Omega(f_2(x_i))$$

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

when, $T \Rightarrow$ no. of leaf nodes

$w \Rightarrow$ weight of leaf

$\gamma, \lambda \Rightarrow$ regularization Parameter

Derivation

Stage I

$$\boxed{m_1} \text{ mean} \rightarrow L = \sum_{i=1}^n L(y_i, \hat{y}_i)$$

$$f_1(x_i)$$

Stage II

$\boxed{m_1}$ $\boxed{m_2}$
mean dtree

$$L = \sum_{i=1}^n L(y_i, \hat{y}_i) + \Omega(f_2(x_i))$$

$f_1(x_i)$ $f_2(x_i)$

$$\hat{y}_i = f_1(x_i) + f_2(x_i)$$

$$L = \sum_{i=1}^n L(y_i, f_1(x_i) + f_2(x_i)) + \Omega(f_2(x_i))$$

Stage III

$\boxed{m_1}$ $\boxed{m_2}$ $\boxed{m_3}$
mean dt₁ dt₂

$f_1(x_i)$ $f_2(x_i)$ $f_3(x_i)$

$$L = \sum_{i=1}^n L(y_i, f_1(x_i) + f_2(x_i) + f_3(x_i)) + \Omega(f_3(x_i))$$

Stage t

$\boxed{m_1}$ $\boxed{m_2}$ $\boxed{m_t}$
mean dt₁ dt_t

$f_1(x_i)$ $f_2(x_i)$ $f_t(x_i)$



$$L = \sum_{i=1}^n L(y_i, f_1(x_i) + f_2(x_i) + \dots + f_t(x_i)) + \Omega(f_t(x_i))$$

$$L = \sum_{i=1}^n L(y_i, \hat{y}_i^{<t-1>} + f_t(x_i)) + \Omega(f_t)$$

At any stage t

$$L^{<t>} = \sum_{i=1}^n L(y_i, \hat{y}_i^{<t-1>} + f_t(x_i)) + \Omega(f_t(x_i))$$

we have to find the such output for $f_t(x_i)$ which will minimize the whole objective function

but as our $\Omega(f_t(x_i))$ faces one ~~Taylor Series~~ (zig-zag curve) & we need to approximate the $\sum_{i=1}^n L(y_i, \hat{y}_i^{<t-1>} + f_t(x_i))$

The solution is apply Taylor Series is technique which do the approximation using polynomial.

Taylor Series Equation

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$$

$$x \rightarrow \hat{y}_i^{<t-1>} + f_t(x_i) \quad \cdot \quad a \rightarrow \hat{y}_i^{<t-1>}$$

from derivating our simplified objective function is

$$L^{<t>} \approx \sum_{i=1}^n \left[L(y_i, \hat{y}_i^{<t-1>}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t(x_i))$$



where

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{t-1})}{\partial \hat{y}_i^{t-1}}$$

(gradient)

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{t-1})}{\partial \hat{y}_i^{t-1 2}}$$

(hessian)

we can remove the constant term.

$$L(y_i, \hat{y}_i^{t-1})$$

now, objective function is

$$L^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t(x_i))$$

Now, expand $\Omega(f_t(x_i))$ to $\rightarrow$

$$L^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma I + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Now, replace

 w_j $f_t(x_i)$ with $\rightarrow w_j$ w_j is weight.

$$\sum_{j=1}^T \rightarrow$$

a.

and replace

$$\sum_{i=1}^n \rightarrow \sum_{j=1}^T \sum_{i \in S_j}$$

$$a_1 + a_2 + \dots + a_n = a_3 + a_1 + a_2 + a_4 + \dots + a_n$$



So our objective function becomes.

$$J^{(L)} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i w_j + \frac{1}{2} \sum_{i \in I_j} h_i w_j^2 \right] + \gamma T \phi + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

$$J^{(L)} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i w_j + \frac{1}{2} \sum_{i \in I_j} h_i w_j + \frac{1}{2} \lambda w_j^2 \right] + \gamma T$$

$$L^{(L)} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

differentiate w.r.t. w_j

$$\frac{dJ^{(L)}}{dw_j} = \sum_{i \in I_j} g_i + \left(\sum_{i \in I_j} h_i + \lambda \right) w_j = 0$$

$$w_j = \frac{- \sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \rightarrow \text{gradient}$$

Heisian

We know

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$$

diff. w.r.t. $\hat{y}_i$

$$\begin{aligned} \frac{\partial L}{\partial \hat{y}_i} &= \sum_{i=1}^n (y_i - \hat{y}_i) \\ &= \sum_{i=1}^n (\hat{y}_i^{(t-1)} - y_i) \end{aligned}$$



and

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{<t-D>})}{\partial \hat{y}_i^{<t-D>2}}$$

diff. w.r.t $\hat{y}_i^{<t-D>}$

$$\frac{\partial L}{\partial \hat{y}_i^{<t-D>}} = \sum_{i=1}^n (y_i - \hat{y}_i^{<t-D>}) \Rightarrow \sum_{i=1}^n (y_i^{<t-D>} - \hat{y}_i^{<t-D>})$$

$$\Rightarrow \sum_{i=1}^n 1 //$$

Now put the differentiated values in w_j . It becomes-

$$b = - \sum_{i \in I_j} \frac{\partial L}{\partial \hat{y}_i^{<t-D>}} (y_i - \hat{y}_i^{<t-D>})$$

$$w_j = \frac{\sum_{i \in I_j} 1 + \lambda}{\sum_{i \in I_j} 1 + \lambda}$$

$$w_j = \frac{\sum_{i \in I_j} (y_i - \hat{y}_i^{<t-D>})}{\sum_{i \in I_j} 1 + \lambda}$$

w_j = Sum of residuals.

No. of residuals + λ

* XGBoost Output value of for Regression.



Output Value for Classification

$$W_j = \frac{\text{Sum of residual}}{\sum_{i \in J_j} P_i (1 - P_i) + \lambda}$$

where $P_i \rightarrow$ Predicted probability of previous timestamp.

$$W_j = \frac{\text{Sum of residual}}{\sum_{i \in J_j} P_i (1 - P_i) + \lambda}$$

where $P_i \rightarrow$ Predicted Probability of previous time-stamp.

* Similarity Score

Derivation

We have to find the minimum loss

$$L^{(1)} = \sum_{j=1}^T \left[\left(\sum_{i \in J_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in J_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

$$w_j \rightarrow \frac{-\sum_{i \in J_j} g_i}{\sum_{i \in J_j} h_i + \lambda}$$

$$L^{(2)}(q) = \sum_{j=1}^T \left[\left(\sum_{i \in J_j} g_i \right) \frac{-\sum_{i \in J_j} g_i}{\sum_{i \in J_j} h_i + \lambda} + \frac{1}{2} \left(\sum_{i \in J_j} h_i + \lambda \right) \left(\frac{-\sum_{i \in J_j} g_i}{\sum_{i \in J_j} h_i + \lambda} \right)^2 \right] + \gamma T$$

$$L^{(2)}(q) = \sum_{j=1}^T \left[\frac{-\left(\sum_{i \in J_j} g_i \right)^2}{\sum_{i \in J_j} h_i + \lambda} + \frac{1}{2} \frac{\left(\sum_{i \in J_j} g_i \right)^2}{\sum_{i \in J_j} h_i + \lambda} \right] + \gamma T$$



$$= \frac{T}{\sum_{i=1}^T} \left[-\frac{1}{2} \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] + \gamma T$$

$$\mathcal{L}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Similarity Score

Tree building using Similarity Score

Similarity Score of Parent.

$$\mathcal{L}^{(t)}(q_p) = -\frac{1}{2} \frac{\left(\sum_{i \in I_p} g_i \right)^2}{\sum_{i \in I_p} h_i + \lambda} + \gamma$$

Similarity Score of Child

$$\mathcal{L}^{(t)}(q_c) = -\frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} \right] + 2\gamma$$

When I_L for left node I_R for right node

$$\mathcal{L}^{(t)}(q_p) - \mathcal{L}^{(t)}(q_c) = -\frac{1}{2} \frac{\left(\sum_{i \in I_p} g_i \right)^2}{\sum_{i \in I_p} h_i + \lambda} + \gamma - \left(-\frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} \right] + 2\gamma \right)$$

take $\frac{1}{2}$ common & $\gamma - 2\gamma = -\gamma$

$$L_{split} = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I_p} g_i \right)^2}{\sum_{i \in I_p} h_i + \lambda} \right] - \gamma$$

Loss reduction after the fit //



Density-based spatial clustering of applications with noise (DBSCAN)

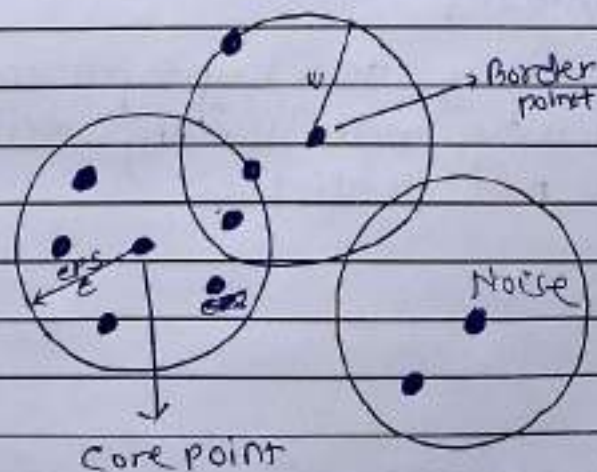
It is a density-based clustering non-parametric algorithm. Given a set of points in some space, it groups together points that are closely packed & marks as outliers points that lie alone in low-density regions.

In this algorithm we have 3 types of data points.

Core Point: A point is a core point if it has more than $MinPts$ points within eps .

Border Point: A point which has fewer than $MinPts$ within eps but it is in the neighborhood of a core point.

Noise or outlier: A point which is not a core or border point.



Steps in Algorithm.

Step 1: Identify all points as either core point, border point or noise point.

Step 2: for all of the unclustered core points

a: Create new cluster

b: add all the points that are unclustered & density connected to the current point into this cluster.

Step 3: for each unclustered border point assign it to the cluster of nearest core point

Step 4: Leave all the noise points as it is.

* Advantages

1. Robust to outliers
2. No need to specify clusters
3. Can find arbitrary shaped clusters
4. Only 2 hyperparameter to tune

* Disadvantages

1. Sensitivity to hyperparameters
2. Difficulty with varying density clusters
3. Does not predict



Syntax Parameters

```
class sklearn.cluster.DBSCAN (eps=0.5, *,  
min_samples=5, metric='euclidean', metric-  
params=None, algorithm='auto', leaf-size=30,  
p=None, n_jobs=None)
```

metric : str, or callable, default='euclidean'

algorithm : {'auto', 'ball-tree', 'kd-tree', 'brute'}

* Implementation

```
>> from sklearn.cluster import DBSCAN  
>> cluster = DBSCAN (eps=3, min_samples=2)  
>> cluster.fit(X)  
>> cluster.labels_
```



Imbalanced data

Imbalanced data is a common problem in ML, which brings challenges to feature correlation, class separation & evaluation & result in poor model performance.

It can be found in countless of applications in finance, healthcare, and public sectors.

Example

bank employee responsible for detecting the validity of credit card transaction.

a) Normal

b) Fraudulent

most transaction are normal - but 0.1% are likely to be fraudulent

Normal (0) = 99%

Fraudulent (1) = 0.1%

this is imbalanced data.

* Challenges with Imbalanced data,

- ① The majority class - class with highest no of samples.
- ② The minority class - class with the lowest no of samples.
- ③ The class ratio - ratio betⁿ minority class & majority class

* Sampling techniques to handle imbalanced data.

1. Random Under-Sampler (RU)

Down samples the majority class by randomly selecting instances to match the minority class.



Pros : $\rightarrow$ Reduction in bias

$\rightarrow$ Faster training

Cons : = Info. loss leading to underfitting

= Sampling Bias

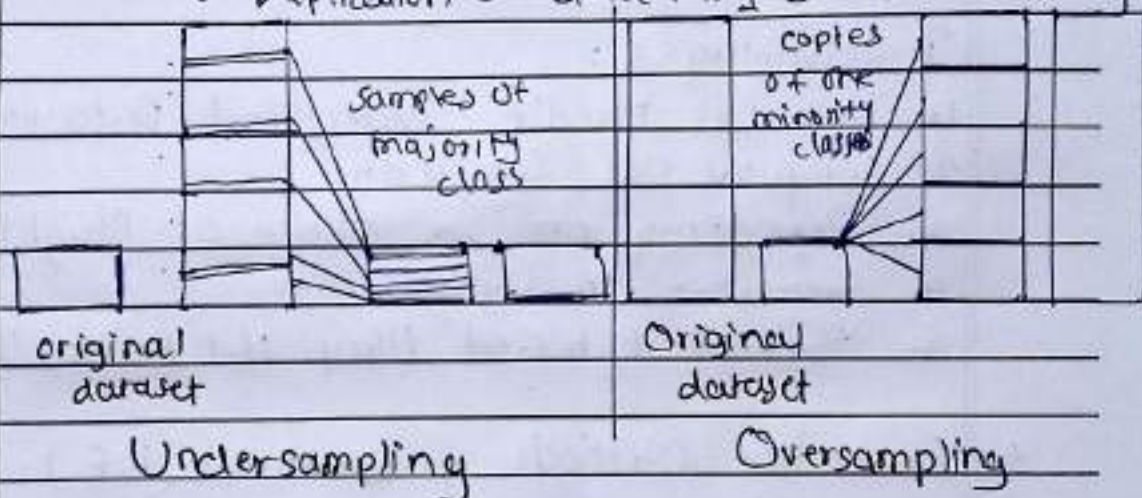
2. Random Over-Sampler (RO)

Upsamples the minority class by randomly duplicating instances until class size are balanced.

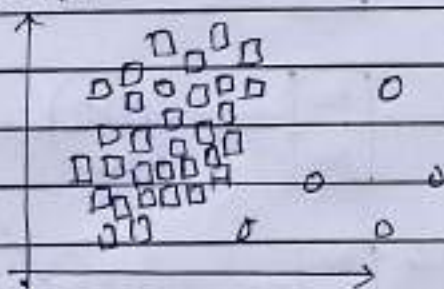
Pros : $\rightarrow$ Avoids data deletion / Reduce bias

Cons : $\rightarrow$ Increased size

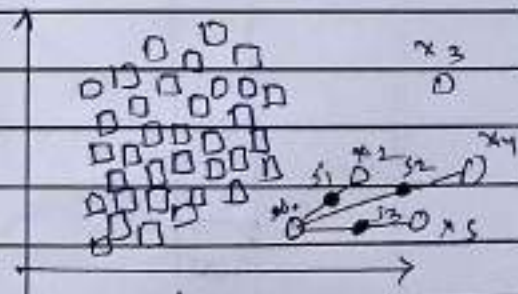
$\rightarrow$ Duplication of data may cause over fitting.



3. SMOTE



a) class imbalance



b) SMOTE

Steps to apply SMOTE

Approach: Generates new instances for the minority class to avoid exact duplication.



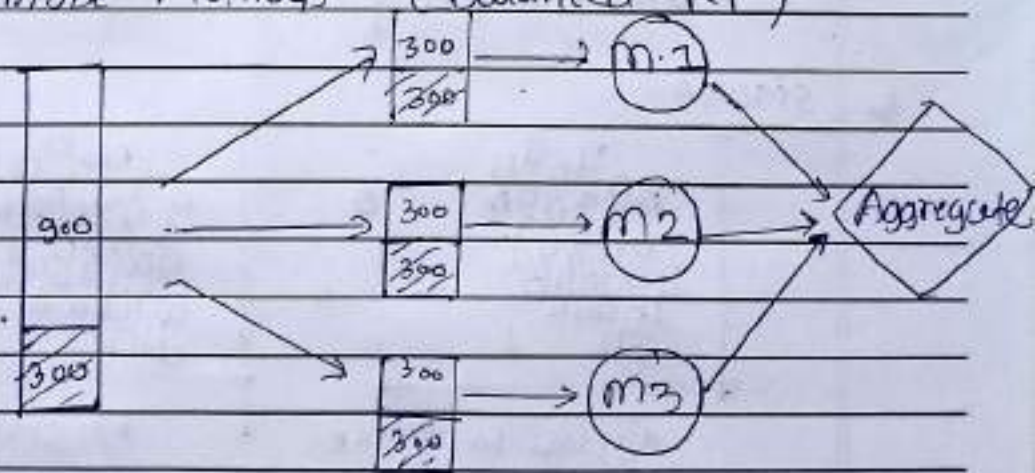
Algorithm:

- step 1: Train KNN on minority class obs. And each obs 5 closest neighbours.
- step 2: Select examples from minority class at random
- step 3: Select a neighbour of each ex. at random
- step 4: Extract a random no. between 0 & 1
- step 5: Cal the new ex. as = original sample
- Factor * (original sample - neighbour)
- step 6: The final dataset consist of the original dataset + newly created examples.

Disadvantages:

1. Does not handle categorical data well
2. Computational Complexity
3. Dependency on one choice of Neighbors
4. Sensitive to Outliers
5. Balance Achieved May Not Reflect True Nature

4. Ensemble Methods (Balanced RF)



12.00

Approach: Modifies the traditional RF algo to handle class imbalance by using balanced bootstrapping.



Algorithm.

Step 1: Each tree is trained on a bootstrap sample that contains an equal no. of instances from each class.

Step 2: This is done by randomly under-sampling the majority class &/or over-sampling the minority class.

5. Cost Sensitive Learning

It is an approach within ML that considers varying costs associated with different types of errors. This method diverges from traditional approaches by introducing a cost matrix, explicitly specifying the penalties like class weight.

Class weights in Cost-Sensitive Learning
class weights adjust the importance of different classes in imbalanced datasets:

It increases model sensitivity to minority class by assigning higher weights to them.

Implementation

1. Manual: Set weights manually based on class importance.
2. Automatic: Use library features like `sklearn`. There is a `class_weight` parameter for every differentiable loss function models. "class_weight" for automatic weight calculation.